

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

NGUYỄN DƯƠNG HOÀNG DUY

KHÓA LUẬN TỐT NGHIỆP
SỬ DỤNG KỸ THUẬT PHÂN TÍCH CHUỖI MÃ LỆNH
ĐỂ PHÁT HIỆN PHẦN MỀM ĐỘC HẠI
KHÔNG XÁC ĐỊNH

Using Opcode Sequences Analysis For Unknown Malware Detection

KỸ SƯ NGÀNH KỸ THUẬT PHẦN MỀM

TP. HỒ CHÍ MINH, 2021

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

NGUYỄN DƯƠNG HOÀNG DUY – 16520291

KHÓA LUẬN TỐT NGHIỆP
SỬ DỤNG KỸ THUẬT PHÂN TÍCH CHUỖI MÃ LỆNH
ĐỂ PHÁT HIỆN PHẦN MỀM ĐỘC HẠI
KHÔNG XÁC ĐỊNH

Using Opcode Sequences Analysis For Unknown Malware Detection

KỸ SƯ NGÀNH KỸ THUẬT PHẦN MỀM

GIẢNG VIÊN HƯỚNG DẪN
THS. MAI TRỌNG KHANG

TP. HỒ CHÍ MINH, 2021

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 79/QĐ-ĐHCNTT ngày 01 tháng 02 năm 2021 của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

- | | |
|----------------------------------|-------------|
| 1. TS. Dương Minh Đức | – Chủ tịch. |
| 2. ThS. Huỳnh Hồ Thị Mộng Trinh. | – Thư ký. |
| 3. ThS. Đinh Nguyễn Anh Dũng | – Ủy viên. |

LỜI CẢM ƠN

Lời đầu tiên của, em xin gửi lời cảm ơn trân trọng đến quý thầy cô khoa Công nghệ Phần mềm trường Đại học Công nghệ thông tin – ĐHQG TP. HCM vì đã tận tình trong việc giảng dạy, tạo điều kiện học tập và rèn luyện cho em trong hơn 4 năm sinh viên vừa qua, giúp em có đủ kiến thức nền tảng và khả năng ứng dụng thực tế để hoàn thành Khóa luận tốt nghiệp này.

Em xin gửi lời cảm ơn chân thành đến thạc sĩ Mai Trọng Khang, người đã tận tình hướng dẫn, định hướng và giúp đỡ em trong quá trình thực hiện Khóa luận tốt nghiệp của mình. Những đóng góp và kinh nghiệm quý báu của thầy là tiền đề để bài Khóa luận này đạt được kết quả tốt nhất.

Khóa luận tốt nghiệp là kết quả của sự cố gắng nghiên cứu và phát triển không ngừng của bản thân em trong hơn 4 tháng qua. Tuy nhiên với vốn kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót, em rất mong nhận được những chỉ dẫn và góp ý của quý thầy cô để bài Khóa luận của em được hoàn thiện hơn.

Lời cuối cùng, kính chúc các thầy cô nhiều sức khỏe, thành công và hạnh phúc. Em xin chân thành cảm ơn !

TP. HCM, ngày 15 tháng 01 năm 2021

Sinh viên thực hiện

Nguyễn Dương Hoàng Duy

MỤC LỤC

TÓM TẮT KHÓA LUẬN	13
MỞ ĐẦU.....	14
CHƯƠNG 1. TỔNG QUAN.....	15
1.1. Các công trình nghiên cứu đã công bố	15
1.2. Các vấn đề cần giải quyết và điểm mới của đề tài	15
CHƯƠNG 2. CÁC LÝ THUYẾT VÀ KIẾN THỨC LIÊN QUAN.....	16
2.1. Máy học	16
2.1.1. Học máy không giám sát.....	16
2.1.2. Học máy bán giám sát.....	16
2.1.3. Học củng cố.....	16
2.1.4. Học máy có giám sát.....	16
2.2. Thuật toán phân lớp trong máy học.....	17
2.2.1. Khái niệm và các thuật toán phân lớp	17
2.2.2. Các phương pháp đánh giá mô hình.....	22
2.3. Lý thuyết đồ thị	25
2.3.1. Định nghĩa.....	25
2.3.2. Biểu diễn đồ thị	25
2.3.3. Thuật toán tìm kiếm trên đồ thị.....	27
2.4. Cấu trúc tập tin Portable Executable	29
2.4.1. MS-DOS Header	30
2.4.2. PE Header.....	32
2.4.3. Section Table.....	32

CHƯƠNG 3. XÂY DỰNG MÔ HÌNH MÁY HỌC.....	36
3.1. Tổng quan	36
3.2. Thu thập dữ liệu	36
3.2.1. Bộ dữ liệu mã độc.....	36
3.2.2. Bộ dữ liệu mã sạch	37
3.2.3. Trích xuất và đồng bộ hóa bộ dữ liệu.....	38
3.3. Rút trích đặc trưng.....	40
3.4. Giảm chiều dữ liệu	45
3.5. Xây dựng mô hình máy học.....	45
3.6. Hiệu chỉnh siêu tham số.....	45
CHƯƠNG 4. ĐÁNH GIÁ HỆ THỐNG	60
4.1. Đánh giá mô hình	60
4.2. Ứng dụng minh họa.....	61
4.2.1. Thu thập bộ dữ liệu.....	61
4.2.2. Huấn luyện mô hình	62
4.2.3. Cài đặt mô hình cho ứng dụng	62
4.2.4. Quét các tập tin.....	63
4.2.5. Quét các tiến trình	64
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	66
6.1. Kết luận.....	66
6.2. Hạn chế và hướng phát triển.....	66
6.2.1. Hạn chế	66
6.2.2. Hướng phát triển.....	66
TÀI LIỆU THAM KHẢO	67

DANH MỤC HÌNH

Hình 2.1 Minh họa ma trận quyết định ^[11]	23
Hình 2.2 Minh họa ROC và AUC ^[11]	24
Hình 2.3 Các loại đồ thị ^[12]	25
Hình 2.4 Biểu diễn đồ thị dạng ma trận kề ^[12]	26
Hình 2.5 Biểu diễn đồ thị dạng danh sách cạnh ^[12]	26
Hình 2.6 Biểu diễn đồ thị dạng danh sách kề ^[12]	27
Hình 2.7 Đồ thị và đường đi ^[12]	28
Hình 2.8 Cây DFS ^[12]	28
Hình 2.9 Cây BFS ^[12]	29
Hình 2.10 Cấu trúc tập tin PE ^[11]	29
Hình 2.11 Cấu trúc tập tin mspaint.exe.....	30
Hình 2.12 Cấu trúc MS-DOS Header.....	31
Hình 2.13 Ví dụ .text section.....	32
Hình 2.14 Ví dụ .data section.....	33
Hình 2.15 Ví dụ .idata section.....	33
Hình 2.16 Ví dụ .edata section.....	34
Hình 2.17 Ví dụ .rdata section.....	34
Hình 2.18 Ví dụ .reloc section.....	35
Hình 3.19 Các bước xây dựng mô hình máy học.....	36
Hình 3.20 Các bước thu thập bộ dữ liệu.....	36
Hình 3.21 Một phần của một mẫu mã độc trong BIG2015.....	37
Hình 3.22 Ví dụ cấu trúc một lệnh trong capstone.....	39
Hình 3.23 Ví dụ về trích xuất thông tin của một lệnh trong asm.....	39
Hình 3.24 CFG cho chương trình trên.....	43
Hình 3.25 ROC và AUC của KNN.....	49
Hình 3.26 ROC và AUC của SVM.....	49
Hình 3.27 ROC và AUC của Logistic Regression.....	50

Hình 3.28 ROC và AUC của Neural Network	50
Hình 3.29 ROC và AUC của Naïve Bayes	51
Hình 3.30 ROC và AUC của LDA	51
Hình 3.31 ROC và AUC của QDA	52
Hình 3.32 ROC và AUC của Decision Tree.....	52
Hình 3.33 ROC và AUC của Random Forest	53
Hình 3.34 ROC và AUC của Extra Trees.....	53
Hình 3.35 ROC và AUC của AdaBoost	54
Hình 3.36 ROC và AUC của Gradient Boosting.....	54
Hình 4.1 Giao diện tính năng thu thập dữ liệu	61
Hình 4.2 Giao diện tính năng huấn luyện mô hình	62
Hình 4.3 Giao diện cài đặt.....	63
Hình 4.4 Giao diện quét các tập tin.....	64
Hình 4.5 Giao diện quét danh sách tiến trình.....	65

DANH MỤC BẢNG

Bảng 2.1 Các chỉ số đánh giá mô hình phân lớp	23
Bảng 3.2 Các loại thanh ghi trong x86	41
Bảng 3.3 Ví dụ n-gram trên ADN.....	44
Bảng 3.4 Danh sách tham số cho KNN	46
Bảng 3.5 Danh sách tham số cho SVM	46
Bảng 3.6 Danh sách tham số cho Logistic Regression.....	46
Bảng 3.7 Danh sách tham số cho Multi-layer Perceptron	46
Bảng 3.8 Danh sách tham số cho Naive Bayes	47
Bảng 3.9 Danh sách tham số cho LDA.....	47
Bảng 3.10 Danh sách tham số cho QDA.....	47
Bảng 3.11 Danh sách tham số cho Decision Tree.....	47
Bảng 3.12 Danh sách tham số cho Random Forest	47
Bảng 3.13 Danh sách tham số cho Extra Trees.....	48
Bảng 3.14 Danh sách tham số cho Ada Boost.....	48
Bảng 3.15 Danh sách tham số cho Gradient Boosting	48
Bảng 3.16 Kết quả kiểm thử của KNN	55
Bảng 3.17 Kết quả kiểm thử của SVM	55
Bảng 3.18 Kết quả kiểm thử của Logistic Regression.....	56
Bảng 3.19 Kết quả kiểm thử của Neural Network	56
Bảng 3.20 Kết quả kiểm thử của Naïve Bayes.....	56
Bảng 3.21 Kết quả kiểm thử của LDA	57
Bảng 3.22 Kết quả kiểm thử của QDA.....	57
Bảng 3.23 Kết quả kiểm thử của Decision Tree.....	57
Bảng 3.24 Kết quả kiểm thử của Random Forest	58
Bảng 3.25 Kết quả kiểm thử của Extra Trees	58
Bảng 3.26 Kết quả kiểm thử của AdaBoost	58
Bảng 3.27 Kết quả kiểm thử của Gradient Boosting.....	59

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Nội dung
1	PE	Portable Executable
2	0-day	Zero-day
3	AI	Artificial Intelligence
4	KNN	K-Nearest Neighbor
5	SVM	Support Vector Machine
6	MLP	Multi-layer Perceptron
7	LDA	Linear Discriminant Analysis
8	QDA	Principal Component Analysis
9	NB	Naive Bayes
10	DT	Decision Tree
11	RF	Random Forest
12	ROC	Receiver Operating Characteristic Curve
13	AUC	Area Under the Curve
14	DFS	Depth-First Search
15	BFS	Breadth-First Search
16	PE	Portable Executable
17	COFF	Common Object File Format
18	CFG	Control Flow Graph

19	TF	Term Frequency
20	IDF	Inverse Document Frequency
21	PR	Page-Rank
22	CNN	Convolutional Neural Network
23	RVA	Relative Virtual Address

TÓM TẮT KHÓA LUẬN

Khóa luận tốt nghiệp với đề tài “SỬ DỤNG KỸ THUẬT PHÂN TÍCH CHUỖI MÃ LỆNH ĐỂ PHÁT HIỆN PHẦN MỀM ĐỘC HẠI KHÔNG XÁC ĐỊNH” bao gồm 5 chương:

Chương 1: Giới thiệu tổng quan về đề tài, phân tích các hướng nghiên cứu đã có liên quan đến đề tài, những vấn đề còn tồn tại, chỉ ra những điểm mới mà đề tài cần tập trung giải quyết.

Chương 2: Trình bày cơ sở lý thuyết và phương pháp khoa học được sử dụng trong đề tài.

Chương 3: Trình bày chi tiết kỹ thuật và phương pháp nghiên cứu được sử dụng trong đề tài.

Chương 4: Đánh giá kết quả đạt được và tính hiệu quả của sản phẩm.

Chương 5: Kết luận, trình bày những vấn đề còn tồn đọng và hướng phát triển trong tương lai.

MỞ ĐẦU

Trong bối cảnh hiện nay, các phần mềm độc hại được tạo ra và phát triển với số lượng ngày càng tăng về số lượng cũng như biến thể. Chúng được áp dụng nhiều kỹ thuật mới và ngày càng tinh vi hơn nhằm qua mặt các hệ thống phòng chống mã độc tân tiến.

Tuy nhiên hầu hết các hệ thống này hiện nay đều dựa trên cơ sở đặc trưng của các mẫu mã độc đã được phát hiện và phân tích. Các nhà phát triển mã độc có thể lợi dụng điều này để phát triển mẫu mã độc khai thác các lỗ hổng Zero-day (0-day) hoặc chỉnh sửa mẫu mã độc cũ nhằm tạo ra biến thể mới với những dấu hiệu hoàn toàn khác nhằm đánh lừa các hệ thống dạng này.

Có nhiều nghiên cứu cho thấy việc sử dụng phân đoạn mã lệnh có khả năng phát hiện được các mẫu mã độc mới mà vẫn đạt hiệu quả cao

Luận văn tập trung vào việc tìm hiểu các kỹ thuật phân tích mã độc ở dạng tập tin Portable Executable (PE) trên hệ điều hành Windows với kiến trúc dòng x86, xây dựng bộ đặc trưng dựa trên phân đoạn mã lệnh, xây dựng mô hình máy học để phát hiện được các loại mã độc không xác định. Đồng thời, nhóm tác giả xây dựng một ứng dụng phân tích và phát hiện mã độc sử dụng mô hình máy học.

CHƯƠNG 1. TỔNG QUAN

1.1. Các công trình nghiên cứu đã công bố

Bài toán phát hiện phần mềm độc hại sử dụng các thuật toán máy học không phải là vấn đề mới. Đối với bài toán “phát hiện phần mềm độc hại không xác định” theo hướng tiếp cận là sử dụng “kỹ thuật phân tích chuỗi mã lệnh” đã được các nhà nghiên cứu công bố một số giải pháp khả thi.

Santos ^[1] và các cộng sự sử dụng đặc trưng term-frequency (TF) cho mỗi mã lệnh và thuật toán học máy bán giám sát Learning with Local and Global Consistency – LLGC.

Sun, Zhi ^[2] và các cộng sự sử dụng đặc trưng term-frequency-inverse document frequency (TF-IDF) kết hợp phương pháp chọn ra k cặp mã lệnh có tần suất xuất hiện nhiều nhất để huấn luyện với các thuật toán máy học kinh điển.

Manavi, Farnoush ^[3] và các cộng sự xây dựng ma trận kề biểu diễn các cặp lệnh n-gram (với $n = 2$) và áp dụng thuật toán tiến hóa (Evolutionary algorithms) trên đặc trưng thu được.

Anh Viet Phan ^[4] và các cộng sự xây dựng Control Flow Graph (CFG) để phân lớp các chương trình vào 4 bài toán khác nhau, sử dụng Convolutional Neural Network (CNN).

1.2. Các vấn đề cần giải quyết và điểm mới của đề tài

Đề tài tập trung giải quyết các vấn đề sau:

- Xây dựng bộ dữ liệu mã độc trích xuất từ bộ dữ liệu BIG2015 với định dạng đơn giản và gọn nhẹ hơn.
- Xây dựng Control Flow Graph với một số cải tiến để đảm bảo luồng chạy thông suốt của chương trình.
- Đánh giá khả năng của nhiều loại phương pháp máy học trên các đặc trưng mới xây dựng.
- Hiện thực ứng dụng với tính năng tự động huấn luyện và chọn lọc đặc trưng.

CHƯƠNG 2. CÁC LÝ THUYẾT VÀ KIẾN THỨC LIÊN QUAN

2.1. Máy học

Machine Learning là một lĩnh vực nhỏ của Khoa Học Máy Tính, và là tập con của trí tuệ nhân tạo - Artificial Intelligence (AI), cung cấp cho máy tính khả năng học mà không cần lập trình rõ ràng. Ngày nay machine learning đã có nhiều ứng dụng thực tế trong đời sống như giáo dục, y tế, tài chính, hệ thống khuyến nghị, công nghệ tri thức, Internet kết nối vạn vật – IoT,...

2.1.1. Học máy không giám sát

Học máy không giám sát - Unsupervised Learning là thuật toán học máy không biết được nhãn mà chỉ có dữ liệu đầu vào. Thuật toán học máy không giám sát sẽ dựa vào cấu trúc của dữ liệu để thực hiện một công việc nào đó, ví dụ như phân nhóm (clustering) hoặc giảm số chiều của dữ liệu (dimension reduction) để thuận tiện trong việc lưu trữ và tính toán.

- *Clustering*: Clustering dùng để phân nhóm toàn bộ dữ liệu đầu vào thành các nhóm nhỏ dựa trên sự liên quan giữa các dữ liệu trong mỗi nhóm.
- *Association*: Association dùng để khám phá ra một quy luật dựa trên nhiều dữ liệu cho trước, thường được sử dụng trong các hệ thống khuyến nghị.

2.1.2. Học máy bán giám sát

Học máy bán giám sát – Semi-Supervised Learning là thuật toán học máy lai giữa học máy có giám sát và học máy không giám sát. Trong loại thuật toán này, chỉ một phần trong dữ liệu đầu vào được gán nhãn.

2.1.3. Học củng cố

Học máy củng cố - Reinforcement Learning là thuật toán học máy có khả năng giúp hệ thống tự động xác định được hành vi dựa trên hoàn cảnh để đạt được lợi ích cao nhất (maximizing the performment). Hiện tại, học máy củng cố chủ yếu được áp dụng vào Lý Thuyết Trò Chơi - Game Theory.

2.1.4. Học máy có giám sát

Học máy có giám sát - Supervised Learning là thuật toán dự đoán đầu ra của một dữ liệu dựa trên các cặp (dữ liệu, nhãn) đã biết từ trước. Học máy có giám sát là nhóm phổ biến nhất trong các thuật toán học máy.

Thuật toán học máy có giám sát còn được tiếp tục chia nhỏ ra thành hai loại chính.

- *Thuật toán phân loại*: Phân loại – Classification dùng trong trường hợp các nhãn của dữ liệu đầu vào được chia thành một số hữu hạn nhóm, ví dụ như true hoặc false, 0 hoặc 1 hoặc 2.
- *Thuật toán hồi quy*: Hồi quy – Regression dùng trong trường hợp các nhãn của dữ liệu đầu vào không thuộc hữu hạn nhóm mà là các giá trị thực cụ thể.

Đề tài sử dụng hai nhãn dữ liệu là 0 cho các mẫu mã sạch và 1 cho các mẫu mã độc. Với bài toán phát hiện mã độc cần thiết phải kết quả mang tính chất rõ ràng và chính xác nên các thuật toán học máy có giám sát được lựa chọn sử dụng cho đề tài này.

2.2. Thuật toán phân lớp trong máy học

2.2.1. Khái niệm và các thuật toán phân lớp

Bài toán phân lớp – Classification thuộc loại học máy có giám sát. Một bài toán được giải quyết bằng thuật toán phân lớp khi số lượng nhãn của dữ liệu đầu vào là hữu hạn, ví dụ như true hoặc false, hay 0 hoặc 1 hoặc 2. Có nhiều bài toán phân lớp dữ liệu như phân lớp nhị phân – binary, phân lớp đa lớp – multiclass, phân lớp đa biến – multivariate. Đối với đề tài này, nhóm tác giả sử dụng bài toán phân lớp nhị phân với mục đích gán nhãn dữ liệu vào 2 lớp khác nhau (bao gồm nhóm mã sạch và nhóm mã độc).

Các thuật toán phân lớp được sử dụng trong đề tài:

2.1.1.1. K-nearest neighbor

K-nearest neighbor (KNN) là một trong những thuật toán máy học có giám sát đơn giản nhất mà vẫn hiệu quả trong một vài trường hợp trong học máy. Khi huấn luyện, thuật toán này không học một điều gì từ dữ liệu training, mọi tính toán được thực hiện khi nó cần dự đoán kết quả của dữ liệu mới. KNN có thể áp dụng được vào cả hai loại của bài toán máy học có giám sát là classification và regression. KNN còn được gọi là một thuật toán Instance-based hay Memory-based learning.

KNN có ưu điểm là độ phức tạp tính toán của quá trình huấn luyện là bằng 0, việc dự đoán kết quả của dữ liệu mới rất đơn giản. Tuy nhiên, KNN rất nhạy cảm với nhiễu khi K nhỏ, việc tính khoảng cách tới từng điểm dữ liệu trong tập training

sẽ tốn rất nhiều thời gian, đặc biệt là với các cơ sở dữ liệu có số chiều lớn và có nhiều điểm dữ liệu. Với K càng lớn thì độ phức tạp cũng sẽ tăng lên. Ngoài ra, việc lưu toàn bộ dữ liệu trong bộ nhớ cũng ảnh hưởng tới hiệu năng của KNN.

2.3.1.2. Support Vector Machine

Support Vector Machine (SVM) là một thuật toán học có giám sát nổi tiếng, có thể sử dụng cho cả việc Classification và Regression. Machine là bài toán đi tìm mặt phân cách hay siêu phẳng (hyperplane) để phân tách dữ liệu thành các lớp riêng biệt, sao cho khoảng cách từ các điểm dữ liệu đến siêu phẳng tìm được là lớn nhất, đồng nghĩa với việc các điểm dữ liệu an toàn nhất so với mặt phân cách.

Trong quá trình huấn luyện SVM, có vô số các mặt phân cách giúp phân biệt các lớp. Với mỗi mặt phân cách, ta có một bộ phân lớp, Khoảng cách gần nhất từ một điểm dữ liệu tới mặt phân cách ấy được gọi là biên (margin) của bộ phân lớp đó. Với các bài toán mà dữ liệu gần như có thể phân tách tuyến tính (linearly separable) hoặc phân tách phi tuyến tính (nonlinear separable), có những cải tiến khác của SVM để thích nghi với dữ liệu đó.

2.3.1.3. Logistic Regression

Logistic Regression là thuật toán học máy được vay mượn từ xác suất thống kê, được sử dụng để tính khả năng phân loại $[0,1]$ với đầu vào dữ liệu cụ thể, trong đó thuật toán được biểu diễn dựa trên hàm Logistic (hàm sigmoid của logarit tự nhiên).

Mặc dù có tên là Regression, Logistic Regression lại được sử dụng nhiều trong các bài toán phân lớp. Mặc dù Logistic Regression không cần có giả thiết dữ liệu hai lớp là linearly separable, nhưng mô hình này chỉ phù hợp với loại dữ liệu mà hai lớp là gần với linearly separable. Logistic Regression không làm việc được với loại dữ liệu phi tuyến (non-linear).

2.3.1.4. Multi-layer Perceptron

Multi-layer Perceptron (MLP) là mạng nơ-ron nhân tạo – (neural network) có nhiều lớp ẩn (hidden layers) ở giữa. Một neuron nhân tạo (còn được gọi là perceptron) là một hàm biến đổi toán học nhận một hoặc nhiều đầu vào đã được nhân với các giá trị gọi là "weights", cộng các giá trị đó lại với nhau thành một giá trị duy nhất. Sau đó giá trị này được đưa vào một hàm phi tuyến (được gọi là activation function) và kết quả của hàm này chính là đầu ra của neuron.

Người ta đã chứng minh được rằng, luôn luôn tồn tại một Neural Network có khả năng xấp xỉ hầu hết các hàm liên tục với chỉ một hidden layer (với số hidden units đủ lớn và nonlinear activation function phù hợp). Tuy nhiên trên thực tế, việc tìm ra số lượng hidden units và nonlinear activation function nói trên nhiều khi bất khả thi. Thay vào đó, thực nghiệm chứng minh rằng Neural Networks với nhiều hidden layers kết hợp với các nonlinear activation function (đơn giản như ReLU) có khả năng xấp xỉ (khả năng biểu diễn) dữ liệu training tốt hơn.

Khi số lượng hidden layers lớn lên, số lượng hệ số cần tối ưu cũng lớn lên và mô hình sẽ trở nên phức tạp, dẫn đến tốc độ tính toán sẽ bị chậm đi rất nhiều hoặc xảy ra hiện tượng có thể biểu diễn rất tốt dữ liệu training nhưng lại không biểu diễn tốt dữ liệu test (overfitting).

2.3.1.5. Naive Bayes

Naive Bayes (NB) là một thuật toán phân lớp, có thể dự đoán xác suất của một phần tử dữ liệu thuộc vào một lớp là bao nhiêu. Phân lớp Bayes được dựa trên định lý Bayes (định lý được đặt theo tên tác giả của nó là Thomas Bayes). Thuật toán NB tính xác suất cho các yếu tố, sau đó chọn kết quả với xác suất cao nhất với giả định là các yếu tố đầu vào được cho là độc lập với nhau. Đây là một thuật toán mạnh mẽ trong các bài toán: dự đoán với thời gian thực, phân loại văn bản, lọc thư rác,...

Naive Bayes có thời gian training và test rất nhanh do giả sử về tính độc lập giữa các thành phần. Nếu giả sử về tính độc lập được thoả mãn (dựa vào bản chất của dữ liệu), NB được cho là cho kết quả tốt hơn so với SVM và Logistic Regression khi có ít dữ liệu training. NB có thể hoạt động với các vector đặc trưng (feature vector) mà một phần là liên tục (sử dụng Gaussian Naive Bayes), phần còn lại ở dạng rời rạc (sử dụng Multinomial hoặc Bernoulli).

2.3.1.6. Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) là một thuật toán máy học có giám sát, thường dùng để giảm chiều dữ liệu có sử dụng thông tin về nhãn của dữ liệu. Ý tưởng cơ bản của LDA là tìm một không gian mới với số chiều nhỏ hơn không gian ban đầu sao cho hình chiếu của các điểm trong cùng một lớp lên không gian mới này là gần nhau trong khi hình chiếu của các điểm của các lớp khác nhau là khác nhau.

Trong LDA, với bài toán có C classes, số chiều của không gian mới chỉ có thể không vượt quá $(C - 1)$. LDA có giả sử ngầm rằng dữ liệu của các lớp đều tuân

theo phân phối chuẩn và các ma trận hiệp phương sai của các lớp là gần nhau. LDA hoạt động rất tốt nếu các lớp là linearly separable, tuy nhiên, chất lượng mô hình giảm đi rõ rệt nếu các lớp là không linearly separable. Điều này dễ hiểu vì khi đó, chiếu dữ liệu lên phương nào thì cũng bị chồng lấn, và việc tách biệt không thể thực hiện được như ở không gian ban đầu.

LDA có một biến thể là Quadratic Discriminant Analysis (QDA). Trong đó giả định rằng các phép đo được phân phối chuẩn. Tuy nhiên, không giống như LDA, trong QDA không có giả định rằng ma trận hiệp phương sai của mỗi lớp là giống nhau. Để ước tính các tham số cần thiết trong QDA, cần nhiều tính toán và dữ liệu hơn trong trường hợp linearly separable.

2.3.1.7. Decision Tree

Decision Tree (DT) là một mô hình máy học có giám sát, có thể được áp dụng vào cả hai bài toán classification và regression. DT là mô hình hỗ trợ ra quyết định dựa trên đồ thị các điều kiện. Mỗi một nút trong (internal node) tương ứng với một biến, đường nối giữa nó với nút con của nó thể hiện một giá trị cụ thể cho biến đó. Tại mỗi nút, ta sẽ đối chiếu các điều kiện thực tế để quyết định rẽ nhánh nào. Nút lá là quyết định cuối cùng.

Decision Tree học những đặc trưng từ data đầu vào, đầu ra là kết quả với hình dạng cây quyết định. Nghĩa là dễ dàng nhìn thấy đặc trưng của data đầu vào. Các loại phương pháp khác cần công việc tiền xử lý rất nhiều, còn với Decision Tree thì hầu như là không cần công đoạn tiền xử lý. Với các loại phân tích như Neural Network được coi như kiểu model hộp đen thì DT giống như model hộp trắng. Tuy nhiên nếu một điểm dữ liệu mới rơi vào nhầm nhánh ở ngay những lần phân chia đầu tiên, kết quả cuối cùng sẽ khác đi rất nhiều. Việc rơi vào nhầm nhánh này rất dễ xảy ra trong trường hợp thuộc tính liên tục được chia thành nhiều nhóm nhỏ, vì hai điểm có thuộc tính tương ứng rất gần nhau có thể rơi vào hai nhóm khác nhau.

2.3.1.8. Random Forest

Random Forest (RF) là một thành viên trong họ thuật toán Decision Tree. Đơn vị của RF là thuật toán cây quyết định, với số lượng hàng trăm. Mỗi cây quyết định được tạo ra một cách ngẫu nhiên từ việc: tái chọn mẫu (bootstrap, random sampling) và chỉ dùng một phần nhỏ tập biến ngẫu nhiên (random features) từ toàn bộ các biến trong dữ liệu. Ở trạng thái sau cùng, mô hình RF thường hoạt động rất chính xác, nhưng đổi lại, ta không thể nào hiểu được cơ chế hoạt động bên trong mô

hình vì cấu trúc quá phức tạp. RF do đó là một trong số những mô hình hộp đen (black box).

Mô hình Random Forest rất hiệu quả cho các bài toán phân loại vì nó huy động cùng lúc hàng trăm mô hình nhỏ hơn bên trong với quy luật khác nhau để đưa ra quyết định cuối cùng. Mỗi mô hình con có thể mạnh yếu khác nhau, nhưng theo nguyên tắc “wisdom of the crowd”, ta sẽ có cơ hội phân loại chính xác hơn so với khi sử dụng bất kì một mô hình đơn lẻ nào.

2.3.1.9. Extra Trees

Extra Trees hay còn gọi là cây cực kỳ ngẫu nhiên là một tập hợp các cây quyết định và các thuật toán có liên quan đến cây quyết định khác như Random Forest. Cây cực kỳ ngẫu nhiên với ý tưởng là tạo ra một lượng lớn cây quyết định từ tập dữ liệu huấn luyện. Quyết định được đưa ra dựa trên giá trị trung bình biểu quyết của các cây quyết định trong trường hợp thuật toán hồi quy hoặc sử dụng biểu quyết đa số trong trường hợp thuật toán phân loại.

Khi triển khai thực tế, Extra Trees có tốc độ nhanh hơn rất nhiều, ổn định hơn mà vẫn đảm bảo độ chính xác gần như tương đương khi so sánh với Random Forest.

2.3.1.10. Ada Boost

Ada Boost là một trong những thuật toán ensemble learning, thuộc loại bagging cùng với Gradient boosting. Ý tưởng của thuật toán này là kết hợp các thuật toán yếu thành một thuật toán mạnh. Kỹ thuật này được thực hiện thông qua việc xây dựng một mô hình từ dữ liệu training, sau đó các mô hình tiếp theo được tạo, cố gắng sửa các lỗi từ mô hình đầu tiên. Các mô hình được thêm vào cho đến khi tập training được dự đoán hoàn hảo hoặc số lượng mô hình đạt ngưỡng cực đại.

AdaBoost được sử dụng với các cây quyết định có độ sâu nhỏ. Sau khi cây đầu tiên được tạo, hiệu suất của cây trên mỗi mẫu huấn luyện được sử dụng làm thông tin để quyết định cây tiếp theo sẽ tập trung vào mẫu huấn luyện nào. Dữ liệu huấn luyện khó dự đoán sẽ được đánh trọng số lớn hơn so với các trường hợp khác. Các mô hình được tạo lần lượt, hiệu suất của mô hình trước sẽ ảnh hưởng đến cách mô hình sau được xây dựng. Sau khi tất cả các cây (mô hình) được xây dựng, dự đoán được thực hiện trên dữ liệu mới. Lần này, mỗi cây được đánh trọng số tùy thuộc vào độ chính xác của nó trên dữ liệu huấn luyện.

2.3.1.11. Gradient Boosting

Gradient Boosting là một kỹ thuật máy học cho các bài toán classification và regression, nhằm tạo ra một mô hình dự đoán mạnh dưới dạng tập hợp các mô hình dự đoán yếu, điển hình là cây quyết định. Gradient Boosting xây dựng mô hình theo kiểu khôn ngoan theo từng giai đoạn giống như các phương pháp Boosting khác, khái quát hóa chúng bằng cách cho phép tối ưu hóa một hàm mất mát có thể phân biệt tùy ý. Gradient Boosting dựa trên giả thuyết rằng mô hình tiếp theo sẽ tốt hơn khi kết hợp với các mô hình trước đó và sẽ giảm thiểu sai số dự đoán tổng thể. Gradient Boosting phân loại dựa trên tốc độ thay đổi – Gradient của hàm. Gradient là một vector trong khi đạo hàm là các giá trị vô hướng – giá trị số.

Gradient Boosting có thể được sử dụng trong lĩnh vực học tập để xếp hạng. Các công cụ tìm kiếm như Yahoo và Yandex sử dụng các biến thể của Gradient Boosting trong các công cụ xếp hạng của họ.

2.2.2. Các phương pháp đánh giá mô hình

Hiệu năng của một mô hình phân lớp thường được đánh giá dựa trên tập dữ liệu kiểm thử (test data). Dữ liệu đầu vào của tập dữ liệu test sau khi được xử lý và huấn luyện qua mô hình, là một vector dự đoán đầu ra (y_{pred}). Ta cần so sánh giữa vector dự đoán y_{pred} này với vector class thật của dữ liệu (y_{true}).

Các chỉ số thường dùng để biểu diễn sự sai khác giữa y_{pred} và y_{true} :

- *True Positive (TP)*: số lượng điểm của lớp positive được phân loại đúng là positive.
- *True Negative (TN)*: số lượng điểm của lớp negative được phân loại đúng là negative.
- *False Positive (FP)*: số lượng điểm của lớp negative bị phân loại nhầm thành positive.
- *False Negative (FN)*: số lượng điểm của lớp positive bị phân loại nhầm thành negative.

		Giá trị thực	
Giá trị dự đoán		Positive (1)	Negative (0)
	Positive (1)	TP	FP

Negative (0)	FN	TN
--------------	----	----

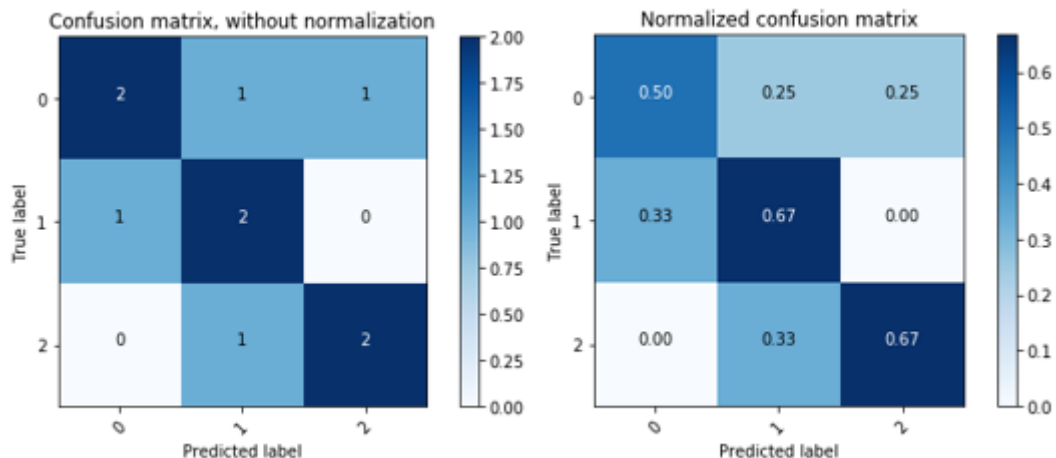
Bảng 2.1 Các chỉ số đánh giá mô hình phân lớp

Các phương pháp đánh giá mô hình được sử dụng phổ biến:

- *Accuracy*: Cách đơn giản và hay được sử dụng nhất là accuracy (độ chính xác). Cách đánh giá này đơn giản tính tỉ lệ giữa số điểm được dự đoán đúng và tổng số điểm trong tập dữ liệu kiểm thử.

$$Accuracy = \frac{TP + TN}{Total}$$

- *Ma trận quyết định – Confusion matrix*: Cách tính sử dụng accuracy như ở trên chỉ cho chúng ta biết được bao nhiêu phần trăm lượng dữ liệu được phân loại đúng mà không chỉ ra được cụ thể mỗi loại được phân loại như thế nào, lớp nào được phân loại đúng nhiều nhất, và dữ liệu thuộc lớp nào thường bị phân loại nhầm vào lớp khác. Để có thể đánh giá được các giá trị này, chúng ta sử dụng một ma trận được gọi là confusion matrix. Về cơ bản, confusion matrix thể hiện có bao nhiêu điểm dữ liệu thực sự thuộc vào một class, và được dự đoán là rơi vào một class.

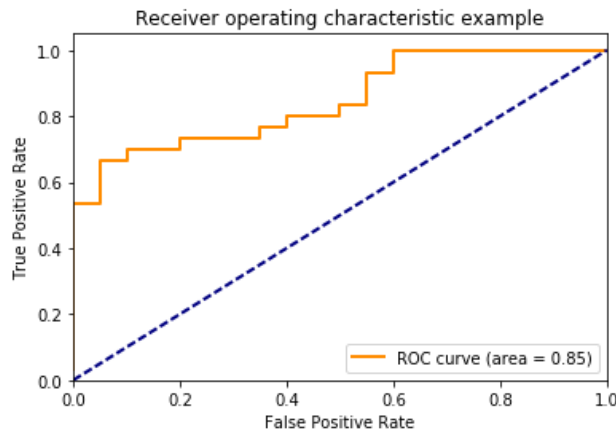


Hình 2.1 Minh họa ma trận quyết định ^[11]

- *Đường cong ROC – Receiver Operating Characteristic Curve (ROC)*: có nguồn gốc từ quân sự, ứng dụng trong việc phát hiện tàu của địch trên màn hình radar. Biểu đồ có trục tung thể hiện tỉ lệ của True Positive, trục hoành

thể hiện tỉ lệ False Positive. Đường biểu diễn càng nằm về phía trên bên trái của biểu đồ thì độ chính xác càng cao.

- *Thông số diện tích bên dưới đường cong ROC – Area Under the Curve (AUC)*: Đại lượng này chính là diện tích nằm dưới ROC curve màu cam. Giá trị này là một số dương nhỏ hơn hoặc bằng 1. Giá trị này càng lớn thì mô hình càng tốt.



Hình 2.2 Minh họa ROC và AUC [11]

- *Precision và Recall*: Với bài toán phân loại mà tập dữ liệu của các lớp là chênh lệch nhau rất nhiều, có một phép đo hiệu quả thường được sử dụng là Precision-Recall. Precision được định nghĩa là tỉ lệ số điểm true positive trong số những điểm được phân loại là positive (TP + FP). Recall được định nghĩa là tỉ lệ số điểm true positive trong số những điểm thực sự là positive (TP + FN). Precision cao đồng nghĩa với việc độ chính xác của các điểm tìm được là cao. Recall cao đồng nghĩa với việc True Positive Rate cao, tức tỉ lệ bỏ sót các điểm thực sự positive là thấp.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

- *F1-score*: là chỉ số trung bình hài hòa - harmonic mean của Precision và Recall.

$$F1 = \frac{2 * (Recall * Precision)}{Recall + Precision}$$

2.3. Lý thuyết đồ thị

2.3.1. Định nghĩa

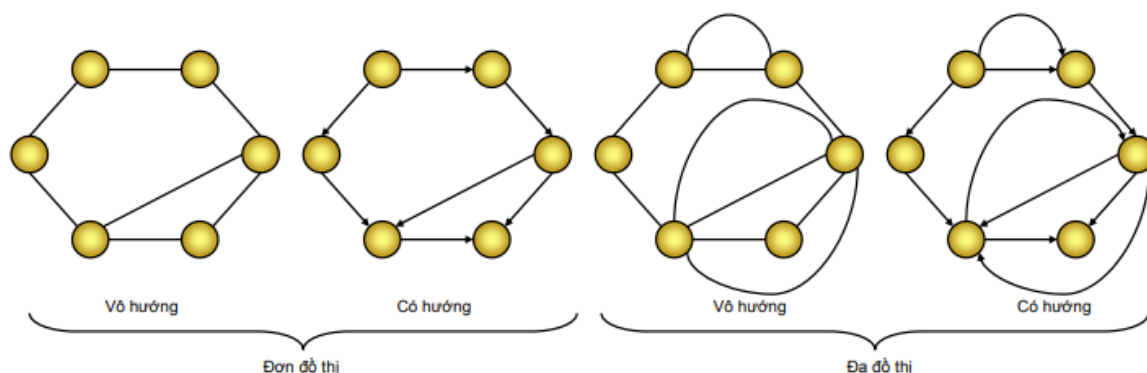
Đồ thị là cấu trúc rời rạc gồm các đỉnh và các cạnh nối tới các đỉnh đó.

$$G = (V, E)$$

Trong đó, V là tập đỉnh (Vertices) và E là tập cạnh (Edges). Cũng có thể coi E là tập các cặp (u, v) với u và v là hai đỉnh thuộc V .

Có thể phân loại đồ thị thành các loại:

- *Đơn đồ thị*: G là đơn đồ thị nếu giữa hai đỉnh u và v bất kỳ thuộc V có tối đa 1 cạnh trong E .
- *Đa đồ thị*: G là đa đồ thị nếu tồn tại hai đỉnh u và v bất kỳ thuộc V có nhiều hơn 1 cạnh trong E .
- *Đồ thị vô hướng*: G là đồ thị vô hướng (undirected graph) nếu tương ứng với hai đỉnh u và v bất kỳ trong V có cạnh nối từ u đến v đều có cạnh nối từ v về u .
- *Đồ thị có hướng*: G là đồ thị có hướng (directed graph) nếu tương ứng với hai đỉnh u và v bất kỳ trong V có cạnh nối từ u đến v nhưng chưa chắc có cạnh nối từ v về u .

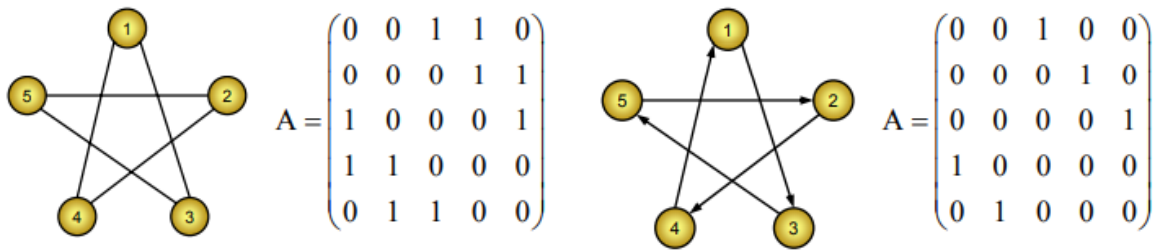


Hình 2.3 Các loại đồ thị ^[12]

2.3.2. Biểu diễn đồ thị

- *Ma trận kề*: Giả sử đồ thị $G = (V, E)$ là một đơn đồ thị với số đỉnh (ký hiệu $|V|$) là n và số cạnh (ký hiệu $|E|$) là m . Ta có biểu diễn đồ thị bằng ma trận vuông $A = [a[i, j]]$ cấp n , trong đó:

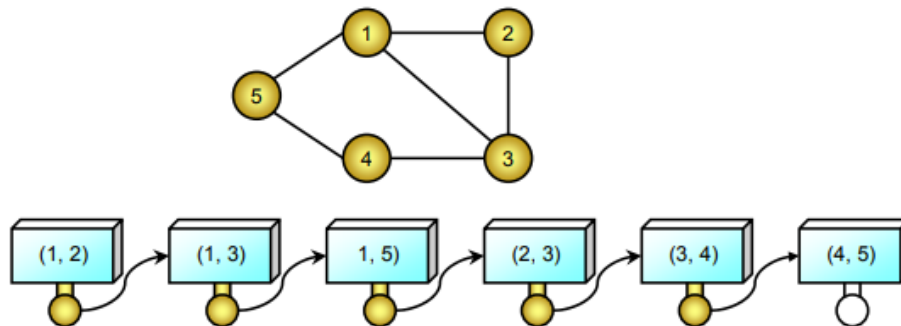
- $a[i,j] = 1$ nếu $(i, j) \in E$
- $a[i,j] = 0$ nếu $(i, j) \notin E$



Hình 2.4 Biểu diễn đồ thị dạng ma trận kề ^[12]

Ma trận kề có ưu điểm là đơn giản, trực quan, độ phức tạp để kiểm tra hai đỉnh có kề nhau hay không là $O(1)$. Tuy nhiên, cách lưu trữ này tốn nhiều bộ nhớ, với $|V|^2$ ô nhớ để biểu diễn cho đồ thị n đỉnh. Trường hợp đỉnh cô lập hoặc có quá ít đỉnh kề, để xét các đỉnh kề với nó đều phải duyệt hết $|V| - 1$ đỉnh còn lại với độ phức tạp là $O(|V|)$ nên gây lãng phí thời gian.

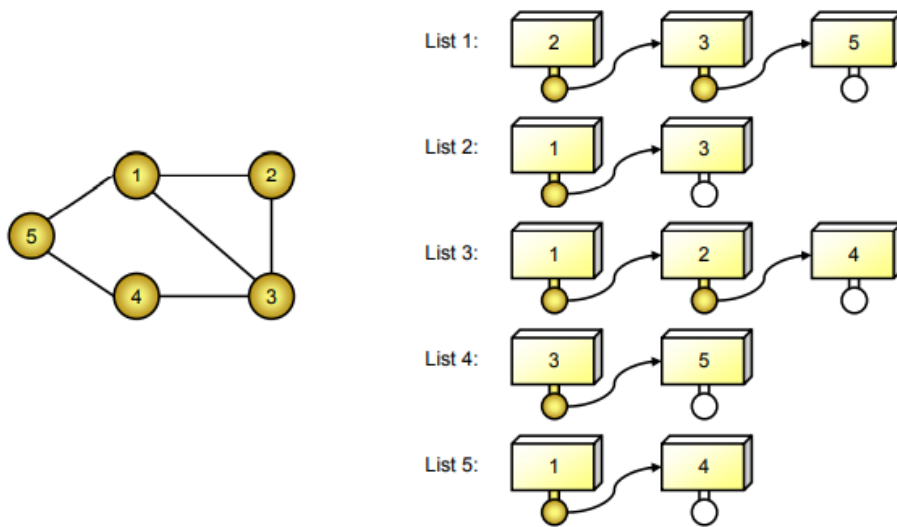
- *Danh sách cạnh*: Với đồ thị $G = (V, E)$, ta có thể biểu diễn đồ thị dưới dạng danh sách cạnh bằng cách liệt kê tất cả các cạnh trong đồ thị vào một danh sách, mỗi phần tử là hai đỉnh (u, v) là một cạnh của đồ thị. Danh sách cạnh được lưu trong bộ nhớ dưới dạng mảng một chiều hoặc danh sách liên kết.



Hình 2.5 Biểu diễn đồ thị dạng danh sách cạnh ^[12]

Cách biểu diễn đồ thị ở dạng danh sách cạnh khá hiệu quả trong trường hợp đồ thị thưa (có số cạnh tương đối nhỏ), và tiết kiệm được không gian lưu trữ ($2 * |E|$ ô nhớ). Dù vậy nhưng danh sách cạnh vẫn còn tồn tại nhược điểm như ma trận kề là cần phải duyệt hết danh sách các cạnh để lọc ra các đỉnh kề với một đỉnh bất kỳ, với độ phức tạp ($O(|E|)$), nên sẽ không hiệu quả trong trường hợp đồ thị dày (số cạnh tương đối nhiều).

- *Danh sách kề*: Xét đồ thị $G = (V, E)$, với mỗi đỉnh u sẽ có danh sách các đỉnh v kề với nó.



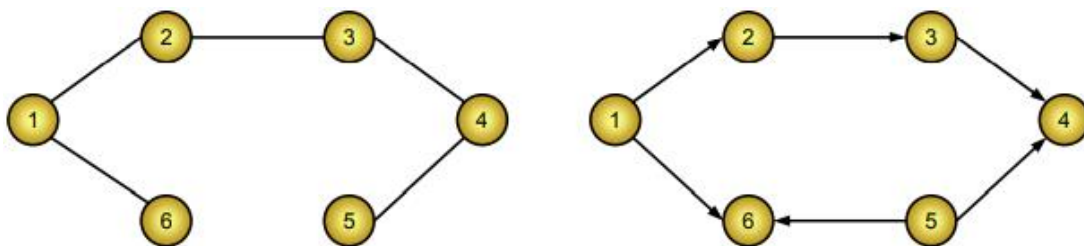
Hình 2.6 Biểu diễn đồ thị dạng danh sách kề [12]

Danh sách kề đã khắc phục được nhược điểm của ma trận kề và danh sách kề, và việc duyệt các đỉnh v kề với đỉnh u cho trước đã dễ dàng và nhanh chóng hơn. Nhược điểm duy nhất của danh sách kề là không thể kiểm tra ngay lập tức hai đỉnh u và v có cạnh nối hay không. Trong trường hợp xấu nhất, thao tác này phải mất đến $O(|V|)$.

2.3.3. Thuật toán tìm kiếm trên đồ thị

Xét đồ thị $G = (V, E)$, u và v là hai đỉnh của G . Đường đi bắt đầu từ u và kết thúc tại v có độ dài l là dãy $(x_0, x_1, \dots, x_l)$ sao cho $x_0 = u$, $x_l = v$, $x_i \in V$, và $(x_i, x_{i+1}) \in E$ và $0 \leq i < l$.

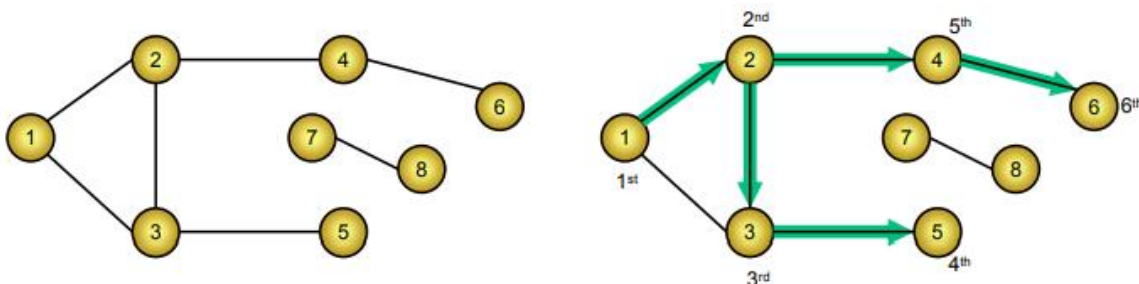
Đỉnh u gọi là đỉnh đầu, đỉnh v gọi là đỉnh cuối. Đường đi có u trùng với v gọi là chu trình (Circuit), đường đi không có cạnh nào đi qua hơn một lần gọi là đường đi đơn, tương tự với chu trình là chu trình đơn.



Hình 2.7 Đồ thị và đường đi ^[12]

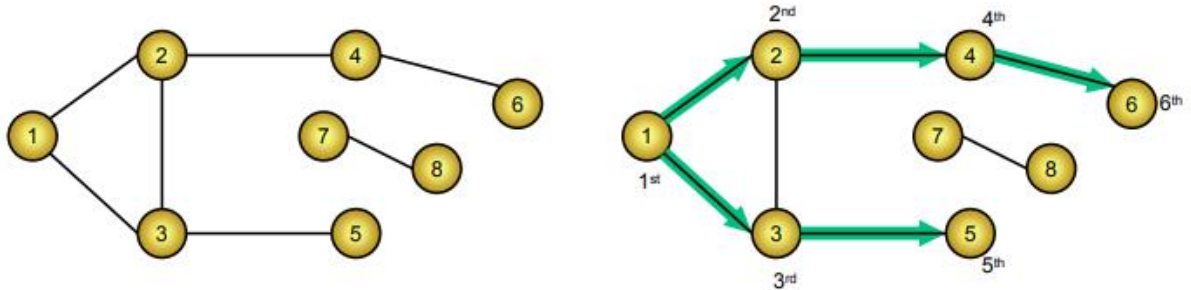
Một trong những bài toán quan trọng nhất trong lý thuyết đồ thị là duyệt các đỉnh có thể đi đến được từ một đỉnh nào đó mà không bỏ sót hay lặp lại đỉnh nào. Thuật toán để giải bài toán dạng này được gọi là thuật toán tìm kiếm trên đồ thị. Có hai thuật toán cơ bản nhất là thuật toán tìm kiếm theo chiều sâu – Depth-First Search (DFS) và thuật toán tìm kiếm theo chiều rộng – Breadth-First Search (BFS).

- *Thuật toán tìm kiếm theo chiều sâu:* Thuật toán dựa trên tư tưởng mọi đỉnh x kề với y sẽ đến được từ y, mọi đỉnh y kề với z sẽ đến được từ z,... Do đó có thể sử dụng kỹ thuật đệ quy hoặc khử đệ quy (như Last-In-First-Out - LIFO) để bắt đầu từ một đỉnh u và duyệt lần lượt các đỉnh còn lại. Thuật toán dừng khi đỉnh cần đến là v được duyệt. Để không đỉnh nào được duyệt quá một lần, ta sử dụng kỹ thuật đánh dấu đỉnh đó để lưu lại và tránh không duyệt lại nó ở những lần đệ quy sau. Sau khi quá trình duyệt kết thúc, ta có thể truy ngược từ đỉnh v trở về u và có được đường đi cần tìm.



Hình 2.8 Cây DFS ^[12]

- *Thuật toán tìm kiếm theo chiều rộng:* Thuật toán dựa trên cơ sở lập lịch cho “mỗi đỉnh” duyệt đến sao cho thứ tự ưu tiên theo “chiều rộng”, tức các đỉnh gần với đỉnh gốc sẽ được duyệt trước. Khi duyệt đến đỉnh x, các đỉnh y kề với x sẽ được thêm vào hàng đợi (thường có cấu trúc First-In-First-Out – FIFO). Cứ thế việc duyệt sẽ lấy từ hàng đợi ra đến khi hàng đợi rỗng thì quá trình duyệt kết thúc.



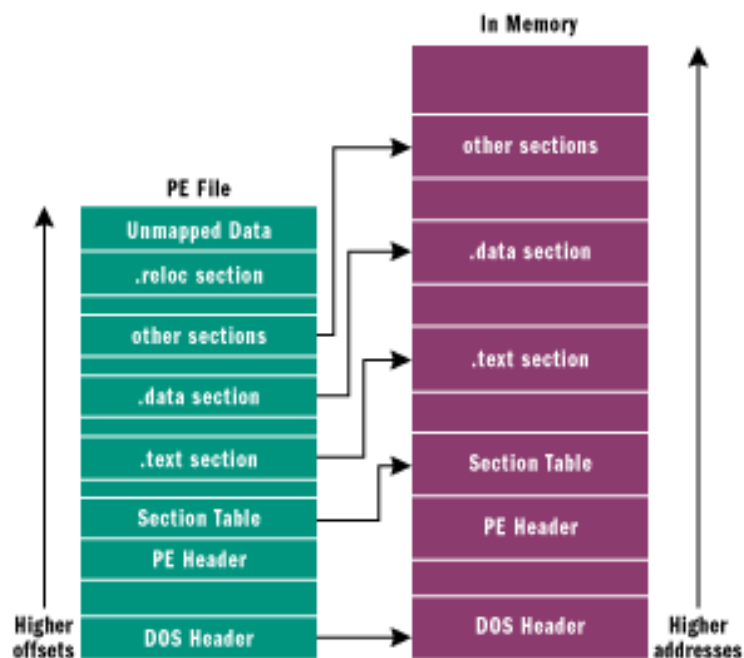
Hình 2.9 Cây BFS ^[12]

Cả hai thuật toán DFS và BFS có độ phức tạp tương đương nhau. Cụ thể trong các cách biểu diễn đồ thị:

- Ma trận kề: $O(|V| + |V|^2) = O(|V|^2)$
- Danh sách cạnh: $O(|V| \cdot |E|)$
- Danh sách kề: $O(|V| + |E|) = O(\max(|V|, |E|))$
-

2.4. Cấu trúc tập tin Portable Executable

Portable Executable (PE) là dạng tập tin có khả năng thực thi linh hoạt trên hệ điều hành Microsoft Windows NT, còn được gọi là Common Object File Format (COFF).



Hình 2.10 Cấu trúc tập tin PE ^[11]

Tập tin PE thường được đặt tên với phần mở rộng như: acm, ax, cpl, dll, drv, efi, exe, mui, ocx, scr, sys, tsp. Một tập tin được gọi là PE khi nó tuân theo cấu trúc COFF được Microsoft quy ước.

File: mspaint.exe - Dos Header - Nt Headers - File Header - Optional Header - Data Directories [x] - Section Headers [x] - Import Directory - Resource Directory - Relocation Directory - Debug Directory - TLS Directory - Address Converter - Dependency Walker - Hex Editor - Identifier - Import Adder - Quick Disassembler - Rebuilder - Resource Editor - UPX Utility	Property	Value
	File Name	C:\Windows\SysWOW64\mspaint.exe
	File Type	Portable Executable 32
	File Info	Microsoft Visual C++ 8
	File Size	722.00 KB (739328 bytes)
	PE Size	722.00 KB (739328 bytes)
	Created	Monday 13 January 2020, 15.41.54
	Modified	Monday 13 January 2020, 15.41.54
	Accessed	Monday 13 January 2020, 15.41.54
	MD5	0642778D34D34B2E04FF795F5FFA50EF
	SHA-1	52AAF5B597915A88761CD72FC67F5FCA1F37CDE4
	Property	Value
	CompanyName	Microsoft Corporation
	FileDescription	Paint
	FileVersion	10.0.18362.207 (WinBuild.160101.0800)
	InternalName	MSPAINT
	LegalCopyright	© Microsoft Corporation. All rights reserved.
	OriginalFilename	MSPAINT.EXE
	ProductName	Microsoft® Windows® Operating System

Hình 2.11 Cấu trúc tập tin mspaint.exe

Cấu trúc cơ bản của một tập tin PE:

2.4.1. MS-DOS Header

MS-DOS Header, hay còn gọi là MS-DOS Stub. Các tập tin PE đều bắt đầu bằng vùng MS-DOS Header. Đây là phân vùng dùng trong trường hợp tập tin PE được chạy trong môi trường MS-DOS, và để hệ điều hành nhận biết đây là tập tin PE. MS-DOS có cấu trúc như sau:

Member	Offset	Size
e_magic	00000000	Word
e_cblp	00000002	Word
e_cp	00000004	Word
e_crlc	00000006	Word
e_cparhdr	00000008	Word
e_minalloc	0000000A	Word
e_maxalloc	0000000C	Word
e_ss	0000000E	Word
e_sp	00000010	Word
e_csum	00000012	Word
e_ip	00000014	Word
e_cs	00000016	Word
e_lfarlc	00000018	Word
e_ovno	0000001A	Word
e_res	0000001C	Word
	0000001E	Word
	00000020	Word
	00000022	Word
e_oemid	00000024	Word
e_oeminfo	00000026	Word
e_res2	00000028	Word
	0000002A	Word
	0000002C	Word
	0000002E	Word
	00000030	Word
	00000032	Word
	00000034	Word
	00000036	Word
	00000038	Word
	0000003A	Word
e_lfanew	0000003C	Dword

Hình 2.12 Cấu trúc MS-DOS Header

Trong đó, trường e_magic chứa dấu hiệu nhận biết PE: “MZ\0\0” (4 byte: 0x4D, 0x5A, 0x00, 0x00). Và e_lfanew chứa độ lệch đến vị trí của PE Header sao với vị trí đầu tập tin.

2.4.2. PE Header

PE Header chứa các thông tin cần thiết cho việc đọc một cấu trúc PE:

- Signature: 4 byte chữ ký của PE: 0x50, 0x45, 0x00, 0x00
- File Header: chứa thông tin lưu trữ vật lý của tập tin.
- Optional Header: chứa thông tin cấu trúc bên trong tập tin PE, ta có thể dựa vào các thông tin này để thêm/xóa/sửa các phân vùng trong PE.

2.4.3. Section Table

Section Table nằm kế tiếp sau PE Header, là một mảng các bản ghi chứa thông tin các phân vùng hiện có trong tập tin PE. Mỗi bản ghi gồm các thông tin về địa chỉ ảo tương đối (Relative Virtual Address - RVA), kích thước phân vùng trên tập tin / trên đĩa, tính chất của phân vùng,... Sau đây là các phân vùng phổ biến có trong cấu trúc PE.

- *Text*: phân vùng .text hay còn gọi là phân vùng .code, là vùng chứa các mã lệnh dùng để thực thi khi chương trình chạy.

```
.text:00586001 loc_586001: ; CODE XREF: sub_585770+8471j
.text:00586001 mov     al, [ecx+36h]
.text:00586004 mov     dl, [ebx+36h]
.text:00586007 cmp     al, dl
.text:00586009 jz      short loc_586035
.text:0058600B cmp     al, 0Ah
.text:0058600D jz      short loc_586029
.text:0058600F cmp     dl, 0Ah
.text:00586012 jz      short loc_586029
.text:00586014 push    0
.text:00586016 push    offset aConflictingOnC ; "conflicting ON CONFLICT clauses specifi"...
.text:00586018 push    [ebp+arg_0]
.text:0058601E call    sub_587F60
.text:00586023 mov     ecx, [ebp+arg_10]
.text:00586026 add     esp, 0Ch
.text:00586029
.text:00586029 loc_586029: ; CODE XREF: sub_585770+89D1j
.text:00586029 ; sub_585770+8A21j
.text:00586029 cmp     byte ptr [ecx+36h], 0Ah
.text:0058602D jnz     short loc_586035
.text:0058602F mov     al, [ebx+36h]
.text:00586032 mov     [ecx+36h], al
```

Hình 2.13 Ví dụ .text section

- *Data*: hay còn gọi là .data, chứa các giá trị khởi tạo cho dữ liệu, nhằm tiết kiệm không gian bộ nhớ cho việc cấp phát và khởi tạo biến trong phân vùng .text. Phân vùng .data thường chỉ chứa các giá trị “read only” như chuỗi, hằng số, ...

```
.data:0049F000 ; Section 2. (virtual address 0009F000)
.data:0049F000 ; Virtual size           : 00006650 ( 26192.)
.data:0049F000 ; Section size in file        : 00004200 ( 16896.)
.data:0049F000 ; Offset to raw data for section: 0009E000
.data:0049F000 ; Flags C0000040: Data Readable Writable
.data:0049F000 ; Alignment           : default
.data:0049F000 ; =====
.data:0049F000
.data:0049F000 ; Segment type: Pure data
.data:0049F000 ; Segment permissions: Read/Write
.data:0049F000 _data          segment para public 'DATA' use32
.data:0049F000          assume cs:_data
.data:0049F000          ;org 49F000h
.data:0049F000 off_49F000      dd offset aBadAllocation
.data:0049F000                                     ; DATA XREF: HEADER:00400120↑o
.data:0049F000                                     ; HEADER:0040021C↑o ...
.data:0049F000                                     ; "bad allocation"
.data:0049F004          align 8
.data:0049F008 dword_49F008      dd 0                ; DATA XREF: sub_424DF6+3E↑r
.data:0049F008                                     ; sub_442680+5D↑o ...
.data:0049F00C dword_49F00C      dd 0                ; DATA XREF: sub_424DF6+38↑r
.data:0049F00C                                     ; sub_4549F1+8↑r ...
.data:0049F010 dword_49F010      dd 0                ; DATA XREF: sub_424DF6+B↑r
... ..
```

Hình 2.14 Ví dụ .data section

- *Import Table*: hay còn gọi là .idata, chứa thông tin về các hàm và thư viện liên kết động được sử dụng bởi tập tin PE.

Module Name	Imports	OFTs	TimeStamp	ForwarderChain	Name RVA	FTs (IAT)
000A485C	N/A	000A332C	000A3330	000A3334	000A3338	000A333C
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
ADVAPI32.dll	14	000A7438	00000000	00000000	000A865C	000A6000
KERNEL32.dll	91	000A75C0	00000000	00000000	000A8BC0	000A6188
GDI32.dll	72	000A749C	00000000	00000000	000A9042	000A6064
OFTs	FTs (IAT)	Hint	Name			
Dword	Dword	Word	szAnsi			
000A8560	000A8560	0264	RegCreateKeyExW			
000A8572	000A8572	0299	RegQueryValueExW			
000A8586	000A8586	025B	RegCloseKey			
000A8594	000A8594	02A9	RegSetValueExW			
.....						

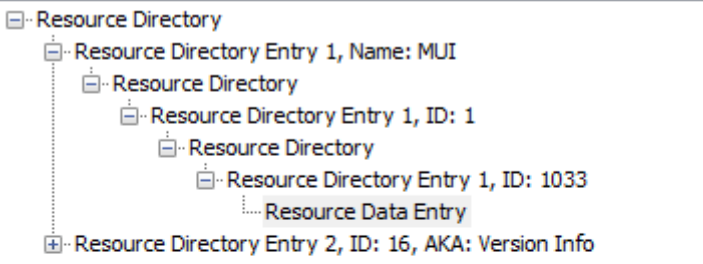
Hình 2.15 Ví dụ .idata section

- *Export Table*: hay còn gọi là .edata, chứa thông tin về các hàm của tập tin PE được trích xuất và cho phép sử dụng bởi chương trình khác.

Ordinal	Function RVA	Name Ordinal	Name RVA	Name
(nFunctions)	Dword	Word	Dword	szAnsi
0000006D	0082C050	0009	01025E5F	ConvertAndEscapePostData
0000006E	0094EE50	000A	01025E78	CreateCoreWebView
0000006F	0063D080	000B	01025E8A	CreateHTMLPropertyPage
00000070	004EC530	000C	01025EA1	DllCanUnloadNow
00000071	0063A230	000D	01025EB1	DllEnumClassObjects
00000072	003F7AD0	000E	01025EC5	DllGetClassObject
00000073	007EBEC0	000F	01025ED7	GetColorValueFromString
00000074	0031DE90	0010	01025EEF	GetWebPlatformObject
00000075	0085D2E0	0011	01025F04	IEIsXMLNSRegistered
00000076	0085D350	0012	01025F18	IERegisterXMLNS
00000077	0040FFB0	0013	01025F28	InitializeLocalHtmlEngine

Hình 2.16 Ví dụ .edata section

- *Resource*: hay còn gọi là .rdata, chứa dữ liệu ở dạng tài nguyên có cấu trúc, được sử dụng bởi tập tin PE cho mục đích lưu trữ.



Member	Offset	Size	Value
OffsetToData	01047680	Dword	0115B448
Size	01047684	Dword	00000130
CodePage	01047688	Dword	00000000
Reserved	0104768C	Dword	00000000

Hình 2.17 Ví dụ .rdata section

- *Relocation*: hay còn gọi là .reloc, có dạng từ điển, chứa các số nguyên với ý nghĩa như độ lệch, nhằm mục đích tính toán lại địa chỉ trong một số trường hợp.

RVA	Size Of Block	Items
01047C00	01047C04	N/A
Dword	Dword	N/A
00001000	00000808	1024
00002000	00000808	1024
00003000	00000740	924
00004000	00000808	1024
00005000	000007C0	920

Item	RVA	Type
Word	N/A	N/A
3000	00001000	HIGHLOW
3004	00001004	HIGHLOW
3008	00001008	HIGHLOW
300C	0000100C	HIGHLOW
3010	00001010	HIGHLOW
3014	00001014	HIGHLOW
3018	00001018	HIGHLOW
301C	0000101C	HIGHLOW
3020	00001020	HIGHLOW
3024	00001024	HIGHLOW
3028	00001028	HIGHLOW

Hình 18 Ví dụ .reloc section

CHƯƠNG 3. XÂY DỰNG MÔ HÌNH MÁY HỌC

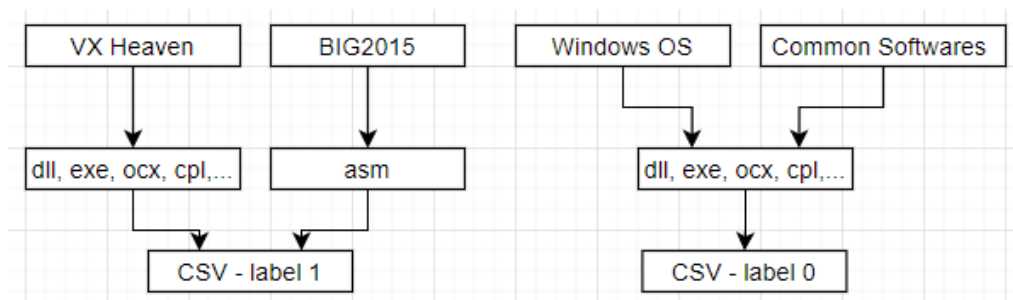
3.1. Tổng quan

Để phát triển được một hệ thống phát hiện mã độc dựa trên chuỗi mã lệnh sử dụng máy học sẽ gồm 4 bước: thu thập dữ liệu, rút trích đặc trưng, xây dựng mô hình máy học và kiểm thử.



Hình 3.19 Các bước xây dựng mô hình máy học

3.2. Thu thập dữ liệu



Hình 3.20 Các bước thu thập bộ dữ liệu

3.2.1. Bộ dữ liệu mã độc

Nhóm tác giả sử dụng hai bộ dữ liệu là “VX Heavens” và “BIG 2015”. Trong đó:

- *VX Heavens*: là một bộ dữ liệu mã độc kinh điển, chứa nhiều loại mã độc với nhiều định dạng khác nhau trên nhiều nền tảng. Trong phạm vi đề tài, ta chỉ quan tâm và lọc ra những mẫu mã độc có định dạng PE và hoạt động được trên hệ điều hành Windows với kiến trúc x86, và thu được 2584 mẫu.
- *BIG 2015*: là bộ dữ liệu mã độc được Microsoft công bố ở cuộc thi “Microsoft Malware Classification Challenge” trên trang chủ của Kaggle. Bộ dữ liệu bao gồm 9 chủng mã độc trên nhiều kiến trúc khác nhau. Tuy nhiên bộ dữ liệu này cung cấp các mẫu ở dạng tập tin assembly (.asm) được phân tích từ IDA (một công cụ nổi tiếng dùng để dịch ngược mã

nguồn của các tập tin thực thi). Nhóm tác giả đã tiến hành phân tích dạng tập tin asm, trích xuất các thông tin cần thiết.

```

52 .text:0040100C          sub_40100C      proc near          ; CODE XREF: start@ANP
53 .text:0040100C 53          push     ebx
54 .text:0040100D 56          push     esi
55 .text:0040100E 57          push     edi
56 .text:0040100F 68 0C 30 40 00  push     offset Buffer ; lpBuffer
57 .text:00401014 68 00 04 00 00  push     400h ; nBufferLength
58 .text:00401019 E8 A0 00 00 00  call     GetTempPathA
59 .text:0040101E E8 A1 00 00 00  call     GetTickCount
60 .text:00401023 50          push     eax
61 .text:00401024 68 0C 30 40 00  push     offset Buffer
62 .text:00401029 68 00 30 40 00  push     offset aSD_dll ; "%s%d.dll"
63 .text:0040102E 68 0C 34 40 00  push     offset LibFileName ; LPSTR
64 .text:00401033 E8 AA 00 00 00  call     wsprintfA
65 .text:00401038 83 C4 10      add     esp, 10h
66 .text:0040103B 6A 0A          push     0Ah ; lpType
67 .text:0040103D 6A 01          push     1 ; lpName
68 .text:0040103F 68 00 00 40 00  push     offset __ImageBase ; hModule
69 .text:00401044 E8 6F 00 00 00  call     FindResourceA
70 .text:00401049 8B D8          mov     ebx, eax
71 .text:0040104B 50          push     eax ; hResInfo
72 .text:0040104C 68 00 00 40 00  push     offset __ImageBase ; hModule
73 .text:00401051 E8 7A 00 00 00  call     LoadResource
74 .text:00401056 8B F8          mov     edi, eax
75 .text:00401058 53          push     ebx ; hResInfo
76 .text:00401059 68 00 00 40 00  push     offset __ImageBase ; hModule
77 .text:0040105E E8 73 00 00 00  call     SizeofResource

```

Hình 3.21 Một phần của một mẫu mã độc trong BIG2015

Sau quá trình thu thập và chọn lọc các mẫu không trùng nhau, nhóm tác giả thu được tổng cộng 18035 mẫu, trong đó 16890 mẫu của BIG2015 và 1145 mẫu của VxHeavens.

3.2.2. Bộ dữ liệu mã sạch

Để mang thực tế, nhóm tác giả thu thập bộ dữ liệu mã sạch từ hai nguồn là các tập tin hệ thống của Windows và các tập tin của một số phần mềm phổ biến:

- *Windows*: sử dụng hệ điều hành Windows 10 1703 64-bit và thu thập các tập tin trong thư mục %Windows%.
- *Phần mềm*: cài đặt các phần mềm phổ biến sau: AIMP, Any Video Converter, Audacity, BlueStacks App Player, Driver Easy, Free Download Manager, FxSound Enhancer, GIMP, Google Chrome, Google Earth Pro, ImgBurn, Internet Download Manager, IrfanView, iTunes, Java 8, Microsoft Edge, Microsoft OneDrive, Microsoft Teams, Mozilla Firefox, Opera, PotPlayer, SHAREit, Skype, TeamViewer, Microsoft Office 365, VLC media player, WhatsApp, Winamp, WinRAR,

WordWeb. Các tập tin nằm trong %ProgramFiles%, %ProgramData%, %UserProfile% được thu thập.

Sau quá trình thu thập và chọn lọc các mẫu không trùng nhau, nhóm tác giả thu được 6013 mẫu mã sạch.

3.2.3. Trích xuất và đồng bộ hóa bộ dữ liệu

Để chuẩn bị cho bước rút trích đặc trưng, dữ liệu đầu vào cần được trích xuất những thông tin cần thiết và đồng bộ về cùng một định dạng.

Với mục đích phát hiện những mẫu mã độc bất kỳ mà không bị ảnh hưởng bởi những đặc trưng thông thường, khó bị đánh lừa bởi các nhà phát triển mã độc, đề tài tập trung trích xuất và phân tích vùng .text trong cấu trúc PE.

Nhóm tác giả sử dụng capstone engine ^[8], một công cụ đa nền tảng có khả năng phân tích tập tin PE trên nhiều loại kiến trúc khác nhau, để phân tích tập tin PE đầu vào.

```

inst = {CsInsn} <CsInsn 0x40f [00a40000000000]: add byte ptr [eax + eax], ah>
  01 addr_size = {int} 4
  01 address = {int} 1039
  01 avx_cc = {int} 0
  01 avx_rm = {int} 0
  01 avx_sae = {bool} False
> 01 bytes = {bytearray: 7} bytearray(b'\x00\xa4\x00\x00\x00\x00\x00')
  01 disp = {int} 0
  01 disp_offset = {int} 3
  01 disp_size = {int} 4
  01 eflags = {int} 63
> 01 groups = {list: 0} []
  01 id = {int} 8
  01 imm_offset = {int} 0
  01 imm_size = {int} 0
  01 mnemonic = {str} 'add'
  01 modrm = {int} 164
  01 modrm_offset = {int} 1
  01 op_str = {str} 'byte ptr [eax + eax], ah'
> 01 opcode = {list: 4} [0, 0, 0, 0]
> 01 operands = {list: 2} [<capstone.x86.X86Op object at 0x0000025A162B56C0>, <capstone.x86.X86Op obj
> 01 prefix = {list: 4} [0, 0, 0, 0]
> 01 regs_read = {list: 0} []
> 01 regs_write = {list: 1} [25]
  01 rex = {int} 0
  01 sib = {int} 0
  01 sib_base = {int} 19
  01 sib_index = {int} 19

```

Hình 3.22 Ví dụ cấu trúc một lệnh trong capstone

Đối với dữ liệu đầu vào là tập tin asm, dữ liệu được phân tách và tổng hợp bằng kỹ thuật lập trình xử lý chuỗi để tổng hợp các byte dữ liệu thuộc phân vùng text, sau đó phân tích bằng capstone như bình thường.



Hình 3.23 Ví dụ về trích xuất thông tin của một lệnh trong asm

Dữ liệu thu thập được sau khi trích xuất có cấu trúc như sau:

- Address: địa chỉ của lệnh (số nguyên 64-bit).
- Opcode value: mã lệnh (số nguyên 8-bit)
- Opcode name: tên của lệnh (chuỗi)
- Operands: danh sách toán hạng (mảng chuỗi)

3.3. Rút trích đặc trưng

Có hai hướng tiếp cận để phân tích một chương trình.

- *Linear*: phân tích các lệnh theo thứ tự địa chỉ từ thấp đến cao.
- *Control Flow Graph (CFG)*: xây dựng đồ thị biểu diễn luồng chạy của chương trình. Thuật toán xây dựng CFG như sau:

Giả sử đồ thị sau khi xây dựng xong là đồ thị có hướng $G = (V, E)$, trong đó V là tập đỉnh, E là tập cạnh. Ban đầu tập đỉnh là rỗng. Xem mỗi lệnh trong chương trình là một đỉnh của đồ thị. Chương trình có n lệnh thì G có $|V| = n + 15$, với 15 đỉnh mới thêm vào là đỉnh đặc biệt X và 14 đỉnh đặc biệt $Y(\text{reg})$ (được đặc tả ngay sau đây). Các cạnh nối được lưu trữ theo cấu trúc danh sách kề.

Đối với lệnh nhảy (*jmp*) là đỉnh có một cạnh nối đến đỉnh đại diện cho địa chỉ lệnh trở đến. Nếu không tồn tại địa chỉ trở đến thì trở đến đỉnh đặc biệt X . Nếu lệnh nhảy sử dụng thanh ghi cho thao tác nhảy thì trở đến đỉnh đặc biệt $Y(\text{reg})$ tùy thuộc vào loại thanh ghi. Trong kiến trúc x86 có 14 loại thanh ghi:

Thứ tự	Loại thanh ghi	Tên thanh ghi trực thuộc	Mô tả
1	Accumulator	eax, ax, ah, al	Dùng cho thao tác I/O, số học
2	Base	ebx, bx, bh, bl	Dùng cho truy cập bộ nhớ
3	Counter	ecx, cx, ch, cl	Dùng cho biến đếm trong vòng lặp và các phép dịch bit
4	Data	edx, dx, dh, dl	Tương tự như Accumulator
5	Stack Pointer	esp, sp, spl	Giữ địa chỉ đỉnh của stack
6	Stack Base Pointer	ebp, bp, bpl	Giữ địa chỉ nền của stack
7	Source Index	esi, si, sil	Dùng cho thao tác sao chép

8	Destination Index	edi, di, dil	Dùng cho thao tác sao chép
9	Stack Segment	ss	Giữ địa chỉ phân vùng stack
10	Code Segment	cs	Giữ địa chỉ phân vùng code
11	Data Segment	ds	Giữ địa chỉ phân vùng data
12	Extra Segment	es	Dùng cho thao con trỏ “far”
13	FS	fs	Dùng cho thao con trỏ “far”
14	GS	gs	Dùng cho thao con trỏ “far”

Bảng 3.2 Các loại thanh ghi trong x86

Đối với lệnh dạng rẽ nhánh (các lệnh rẽ nhánh trong x86: *jno, js, jns, je, jz, jne, jnz, jb, jnae, jc, jnb, jae, jnc, jbe, jna, ja, jnbe, jl, jnge, jge, jnl, jle, jng, jg, jnle, jp, jpe, jnp, jpo, jcxz, jecxz*) là đỉnh có hai cạnh nối trực tiếp đến hai đỉnh: đỉnh đại diện cho lệnh tuyến tính ngay tiếp theo (nếu có) và đỉnh đại diện cho địa chỉ lệnh nhảy trở đến.

Đối với các lệnh *call*, đây là lệnh mang tính chất “khứ hồi” nhờ vào lệnh *ret*. Ta sẽ lưu lại danh sách lệnh *call* để thuận tiện cho việc phân tích lệnh *ret*. Đồng thời, đỉnh đại diện cho lệnh *call* sẽ có một cạnh nối trực tiếp đến đỉnh đại diện cho lệnh *call* trở đến.

Đối với lệnh *ret*, ta sẽ thực hiện kỹ thuật truy vết ngược ở toàn bộ nhánh để tìm ra các lệnh *call* có đường đi đến, sau đó thêm một cạnh nối từ đỉnh *ret* trở đến đỉnh kế tiếp tuyến tính của lệnh *call* đã tìm được (nếu có). Nhóm tác giả lựa chọn thuật toán DFS cho việc truy vết ngược này kết hợp kỹ thuật lỉnh canh nhằm tối ưu hóa số lần duyệt. Đồng thời, nhóm tác giả đề xuất thêm kỹ thuật “truy vết trực tiếp” để có thể truy xuất ngược đến đỉnh nhánh (đỉnh có nhiều hơn một cạnh vào) để tăng tốc quá trình truy vết ở những lần bắt đầu duyệt từ các đỉnh *ret* sau.

Đối với các lệnh còn lại chỉ có một cạnh nối từ nó đến đỉnh đại diện cho lệnh tuyến tính tiếp theo (nếu có).

Như vậy, ngoại trừ lệnh rẽ nhánh có bậc ra tối đa là hai thì các lệnh còn lại đều có bậc ra tối đa là một. Giá trị bậc vào của từng đỉnh luôn thuộc $[0, |V|]$.

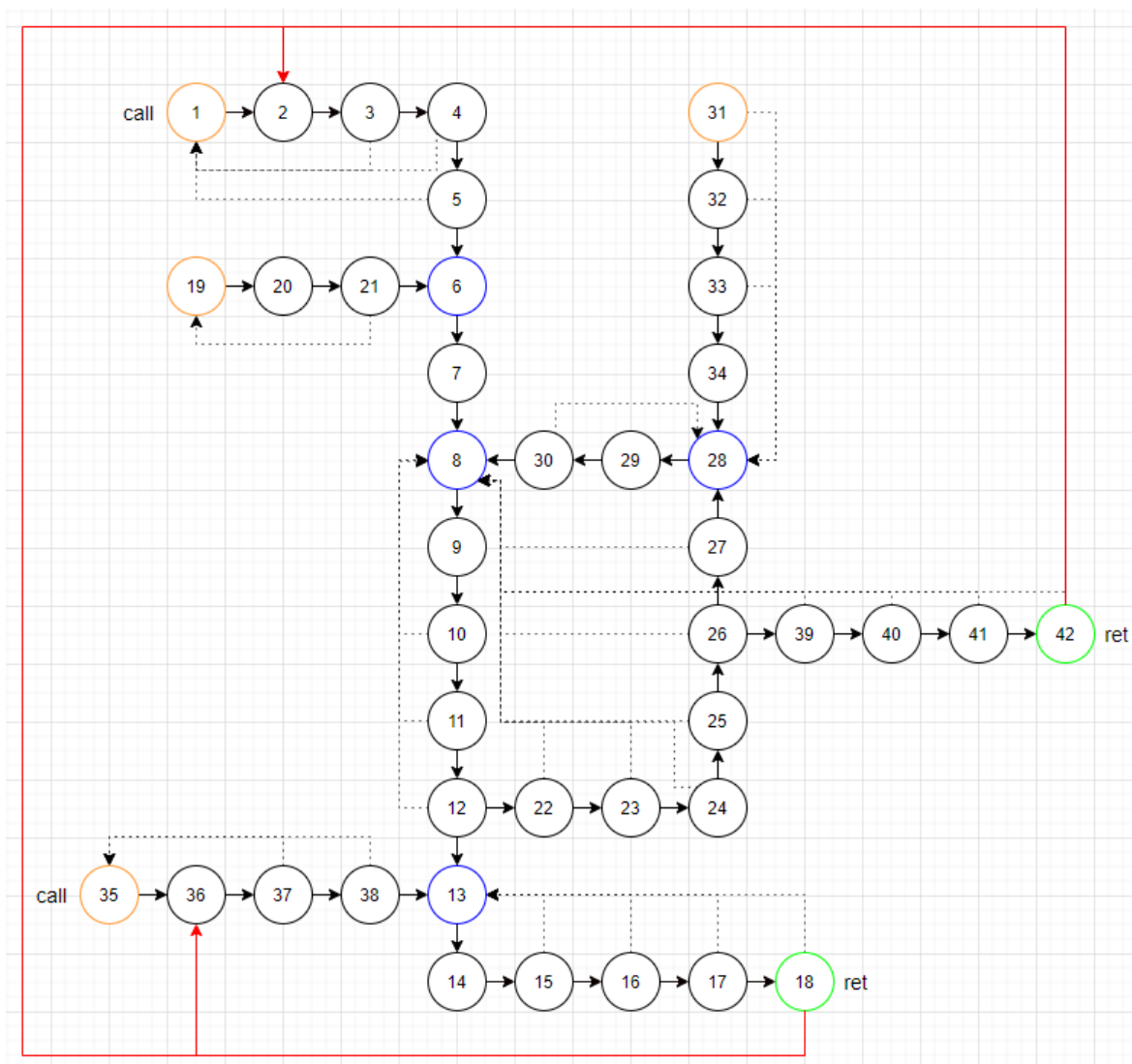
Xét chương trình sau:

```

1          call
2          xor     ebx, ebx
3          cmp     cx, 30h
4          mov     ebp, esp
5          test    ebx, ebx
6  flag_1: inc     ebx
7          pop     edi
8  flag_3: pop     esi
9          xor     eax, eax
10         pop     ebx
11         pop     ebp
12         jz      flag_2
13  flag_6: mov     ecx, 10h
14         lea     ebx, [ecx+ecx*2+10h]
15         push    ebx
16         mov     ebx, 1
17         shl     ebx, 0Ch
18         ret
19         mov     eax, 100h
20         lea     eax, [eax+700h]
21         jmp     flag_1
22  flag_2: pusha
23         push    eax
24         add     esp, 10h
25         mov     ebx, eax
26         je      flag_4
27         mov     ecx, esp
28  flag_5: push    0
29         pop     ec
30         jmp     flag_3
31         pop     edi
32         pop     esi
33         pop     ebx
34         jmp     flag_5
35         call
36         mov     edi, eax
37         push    ebx
38         jmp     flag_6
39  flag_4: push    80h
40         push    2
41         push    0
42         ret

```

Ta thu được CFG như sau:



Hình 3.24 CFG cho chương trình trên

Sau khi thu được thông tin về tập lệnh của các mẫu, ta tiến hành trích xuất đặc trưng dựa trên mã lệnh. Với mỗi mẫu dữ liệu của bài toán này, ta có chuỗi các mã lệnh với miền giá trị nguyên $[0, 270]$ để trích xuất đặc trưng, trong đó 256 giá trị mang ý nghĩa mã lệnh thực tế và 15 giá trị đặc biệt đại diện cho các thanh ghi.

Nhóm tác giả đã tìm hiểu và áp dụng các đặc trưng sau đây:

- *N-gram*: là các chuỗi n phần tử liên tiếp được trích xuất từ một mẫu văn bản. Ví dụ đối với chuỗi DNA “AGCTTCGA”, ta có danh sách các n -gram sau:

Chuỗi ban đầu	1-gram	2-gram	3-gram
AGCTTCGA	A, G, C, T, T, C, G, A	AG, GC, CT, TT, TC, CG, GA	AGC, GCT, CTT, TTC, TCG, CGA

Bảng 3.3 Ví dụ n-gram trên ADN

Với n càng lớn, số lượng chuỗi thu được từ n-gram càng lớn. Số lượng chuỗi không trùng nhau thu được từ n-gram trong tập k ký tự: k^n .

- *Term Frequency (TF)*: là cách đếm số lần xuất hiện của từ trong văn bản. Có nhiều cách tính của TF cho từ t trong văn bản d. Nhóm tác giả sử dụng dạng chuẩn hóa với tần số tăng cường K để hạn chế sự lệch lạc đối với các tài liệu dài (K là 0.5):

$$tf(t, d) = K + K \frac{f(t, d)}{\max(t' \in d) f(t', d)}$$

- *Inverse Document Frequency (IDF)*: là thước đo mức độ phổ biến hay hiếm của từ trên tất cả các tài liệu. Giá sử D là tập toàn bộ văn bản.

$$idf(t, D) = \log \frac{|D|}{1 + |\{d \in D: t \in d\}|}$$

- *Term Frequency–Inverse Document Frequency (TF-IDF)*: là sự kết hợp của TF và IDF.

$$tfidf(t, d, D) = tf(t, d).idf(t, D)$$

- *Page-Rank (PR)*: là thuật toán được sử dụng bởi Google Search để xếp hạng trang web trên kết quả của công cụ tìm kiếm. PR là phương pháp dùng để ước lượng độ quan trọng của từng trang web, dựa trên các liên kết mà trang web được trở tới. Công thức tính Page-Rank cho trang web i với hệ số d (thường là 0.85), M là tập các trang web trở đến i, N là tổng số trang web:

$$PR(i) = \frac{1 - d}{N} + d \sum_{j \in M} \frac{PR(j)}{L(j)}$$

3.4. Giảm chiều dữ liệu

Nhằm giảm thiểu mức độ nhiễu của các giá trị ít xuất hiện trong tập dữ liệu, nhóm tác giả sử dụng các kỹ thuật giảm chiều dữ liệu cho đặc trưng thu được trước khi đưa vào huấn luyện mô hình.

Principal Component Analysis (PCA) là một trong những kỹ thuật giảm chiều dữ liệu phổ biến nhất. Phương pháp này dựa trên quan sát rằng dữ liệu thường không phân bố ngẫu nhiên trong không gian mà thường phân bố gần các đường/mặt đặc biệt nào đó. PCA xem xét một trường hợp đặc biệt khi các mặt đặc biệt đó có dạng tuyến tính là các không gian con (subspace).

3.5. Xây dựng mô hình máy học

Nhóm tác giả sử dụng 14 thuật toán máy học cho bài toán phân lớp có giám sát: K-Nearest-Neighbor, Linear SVM, RBF SVM, Linear Regression, Logistic Regression, Multi-layer Perceptron, Naive Bayes, Linear Discriminant Analysis, Quadratic Discriminant Analysis, Decision Tree, Random Forest, Extra Trees, AdaBoost, Gradient Boosting. Việc đánh giá mô hình huấn luyện dựa vào phương pháp Precision-Recall, F-score, ROC – AUC và cross-validation.

Dữ liệu đầu vào sẽ được phân tách thành hai tập huấn luyện và kiểm thử với tỉ lệ 80% cho việc huấn luyện và 20% cho việc kiểm thử. Với kỹ thuật đánh giá cross-validation, tập dữ liệu được chia thành 5 phần, trong đó 4 phần được sử dụng để huấn luyện mô hình máy học và 1 phần dùng cho việc đánh giá mô hình máy học. Sau khi lặp lại việc huấn luyện 5 lần với cùng tập dữ liệu, kết quả trung bình thu được của mỗi thuật toán được sử dụng làm tiêu chí đánh giá thuật toán. Do tập dữ liệu đang sử dụng có số lượng mẫu thuộc nhãn 1 (18035) nhiều hơn số lượng mẫu thuộc nhãn 0 (6013), nên chỉ số Precision/Recall/F1-score được sử dụng để đánh giá độ chính xác của mô hình máy học thay vì chỉ số chính xác (Accuracy).

3.6. Hiệu chỉnh siêu tham số

Để cải thiện khả năng đánh giá của mô hình, nhóm tác giả tiến hành hiệu chỉnh siêu tham số trên từng bộ phân lớp. Nhằm mục đích phát hiện mã độc không xác định, việc hiệu chỉnh sẽ tối ưu chỉ số Recall.

STT	Tên tham số	Miền giá trị
1	n_neighbors	[1, 20]
2	weights	{"uniform", "distance"}
3	metric	{"euclidean", "manhattan", "minkowski"}

Bảng 3.4 Danh sách tham số cho KNN

STT	Tên tham số	Miền giá trị
1	kernel	{"linear", "poly", "rbf", "sigmoid"}
2	C	{0.001, 0.01, 0.1, 1, 10, 100, 1000}
3	gamma	{0.001, 0.01, 0.1, 1, 10, 100, 1000}

Bảng 3.5 Danh sách tham số cho SVM

STT	Tên tham số	Miền giá trị
1	C	{0.001, 0.01, 0.1, 1, 10, 100, 1000}
2	penalty	{"l1", "l2"}
3	solver	{"newton-cg", "lbfgs", "liblinear"}

Bảng 3.6 Danh sách tham số cho Logistic Regression

STT	Tên tham số	Miền giá trị
1	hidden_layer_sizes	[1, 8]
2	activation	{"identity", "logistic", "tanh", "relu"}

Bảng 3.7 Danh sách tham số cho Multi-layer Perceptron

STT	Tên tham số	Miền giá trị
1	var_smoothing	{1e12, 1e11, 1e10, 1e9, 1e8, 1e7, 1e6}

Bảng 3.8 Danh sách tham số cho Naive Bayes

STT	Tên tham số	Miền giá trị
1	solver	{"svd", "lsqr", "eigen"}
2	shrinkage	{"auto", None} + logspace(-3, 0, 10)

Bảng 3.9 Danh sách tham số cho LDA

STT	Tên tham số	Miền giá trị
1	reg_param	logspace(-3, 0, 30)

Bảng 3.10 Danh sách tham số cho QDA

STT	Tên tham số	Miền giá trị
1	min_samples_split	{0.1, 0.2, 0.3, 0.4, 0.5}
2	min_samples_leaf	{0.1, 0.2, 0.3, 0.4, 0.5}
3	max_features	{"auto", "sqrt", "log2", None}
4	max_depth	{1, 50, 100, None}

Bảng 3.11 Danh sách tham số cho Decision Tree

STT	Tên tham số	Miền giá trị
1	min_samples_split	{0.1, 0.2, 0.3, 0.4, 0.5}
2	max_depth	{1, 50, 100, None}

Bảng 3.12 Danh sách tham số cho Random Forest

STT	Tên tham số	Miền giá trị
1	min_samples_split	{0.1, 0.2, 0.3, 0.4, 0.5}
2	max_depth	{1, 50, 100, None}

Bảng 3.13 Danh sách tham số cho Extra Trees

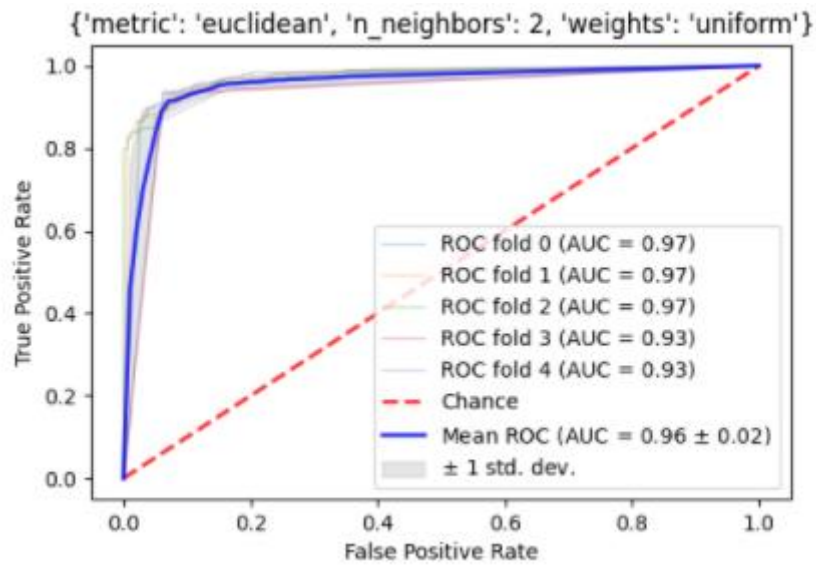
STT	Tên tham số	Miền giá trị
1	learning_rate	{0.0001, 0.001, 0.01, 0.1, 1}
2	algorithm	{"SAMME", "SAMME.R"}

Bảng 3.14 Danh sách tham số cho Ada Boost

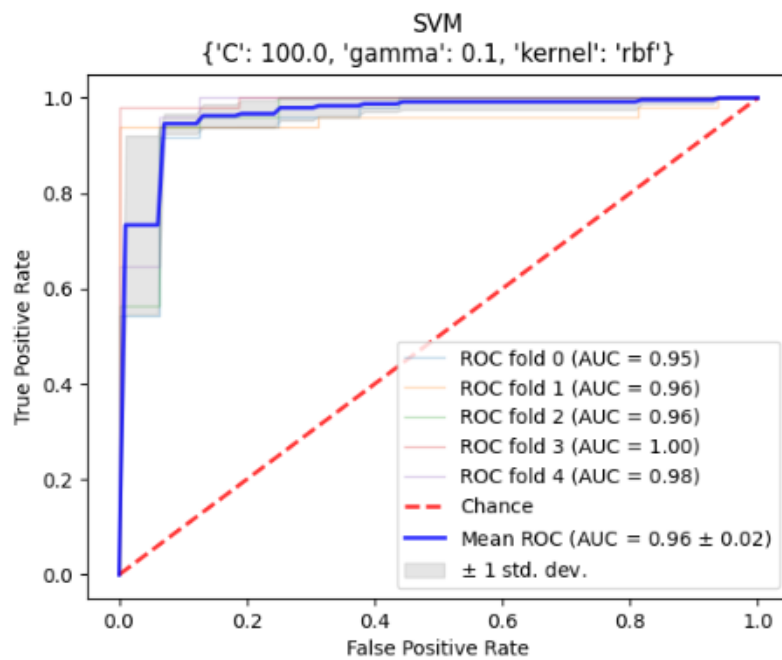
STT	Tên tham số	Miền giá trị
1	loss	"deviance", "exponential"
2	learning_rate	{0.0001, 0.001, 0.01, 0.1, 1}
3	n_estimators	{10, 20, 50, 80, 100}

Bảng 3.15 Danh sách tham số cho Gradient Boosting

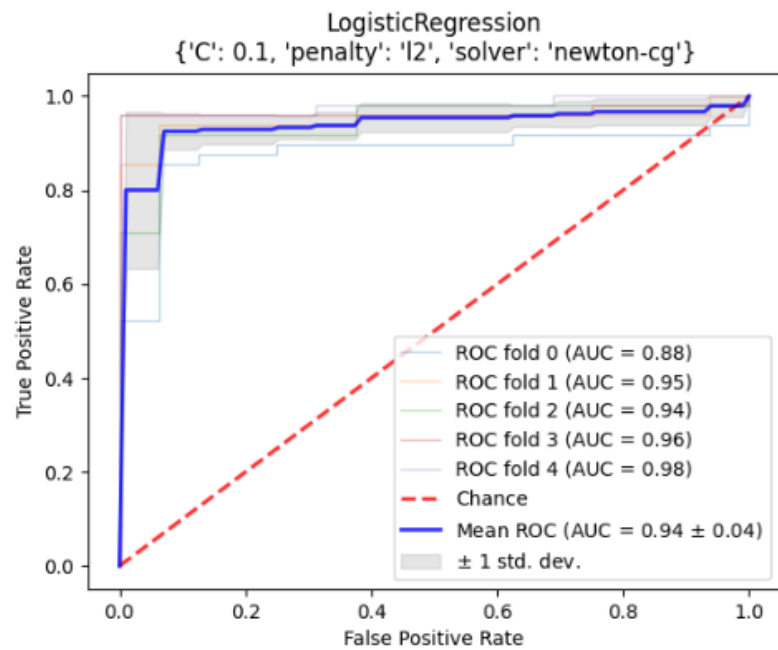
Các biểu đồ thể hiện ROC Curve và AUC trên tập huấn luyện sau khi hiệu chỉnh siêu tham số:



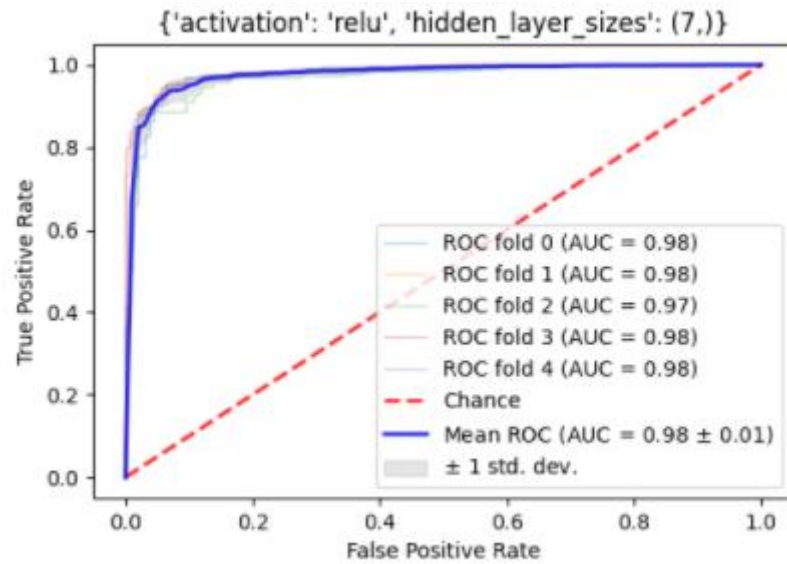
Hình 3.25 ROC và AUC của KNN



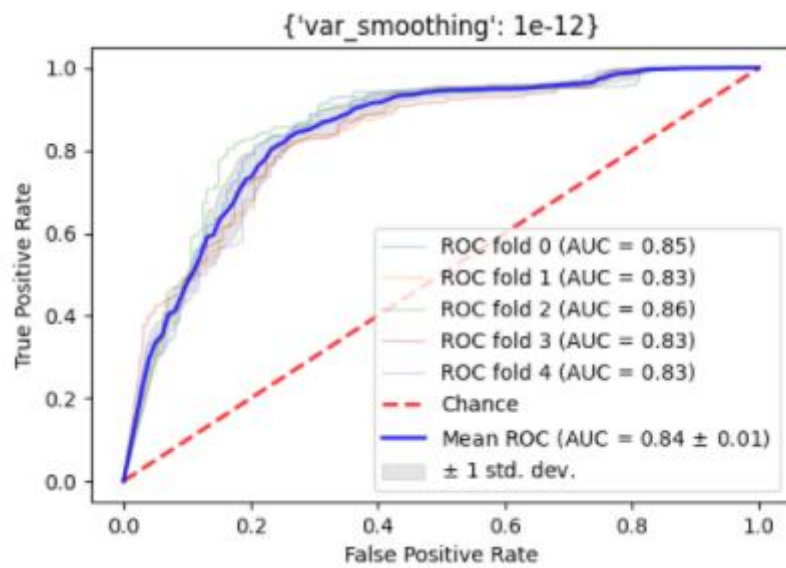
Hình 3.26 ROC và AUC của SVM



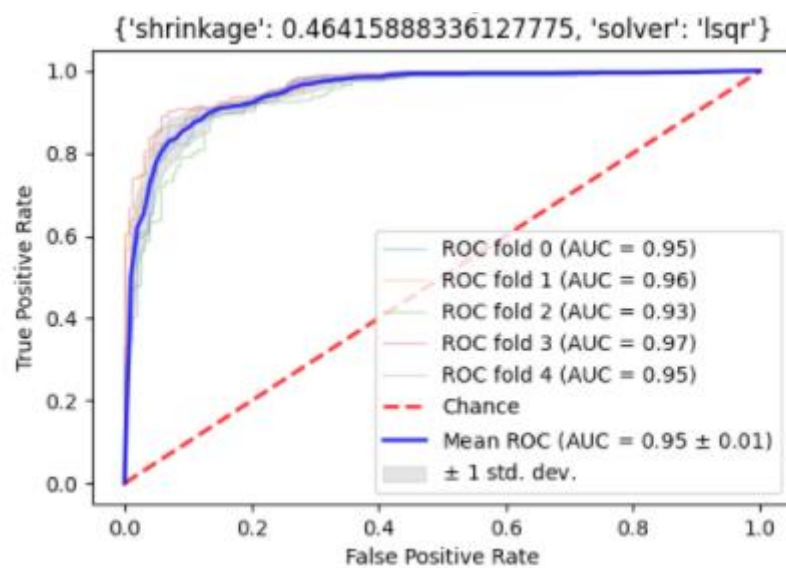
Hình 3.27 ROC và AUC của Logistic Regression



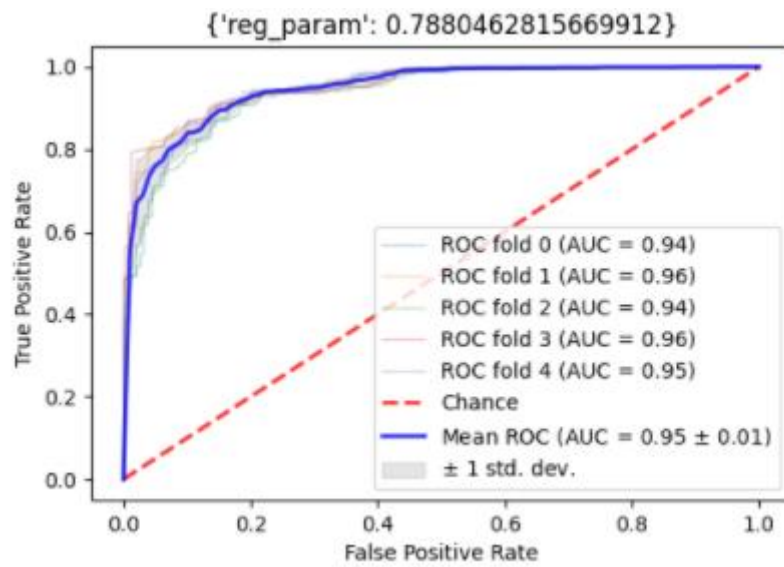
Hình 3.28 ROC và AUC của Neural Network



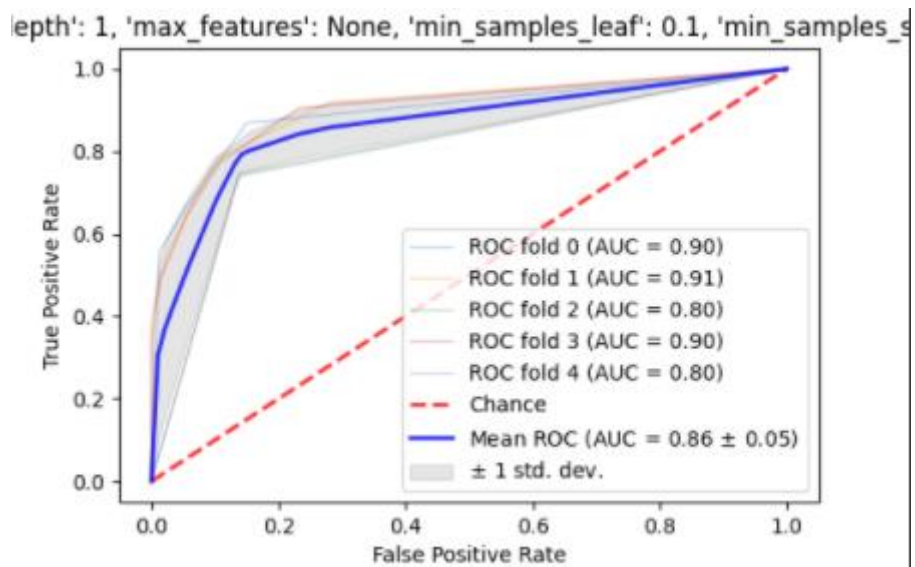
Hình 3.29 ROC và AUC của Naïve Bayes



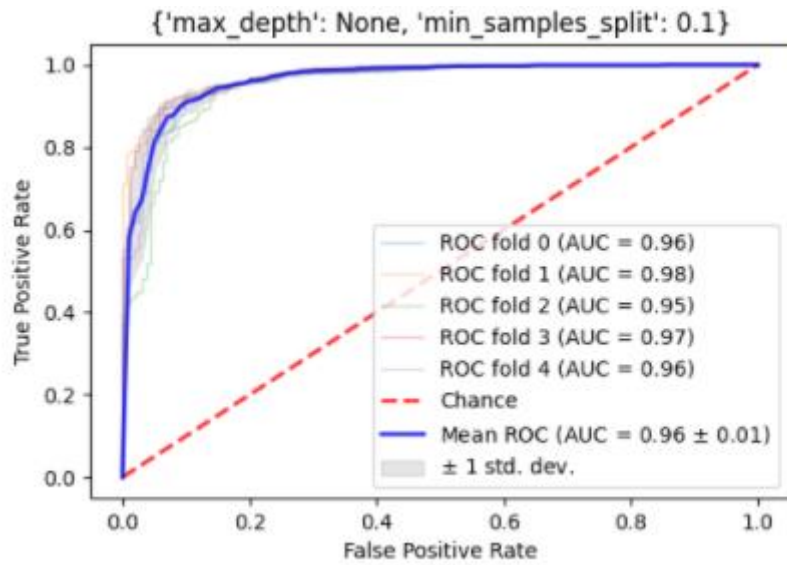
Hình 3.30 ROC và AUC của LDA



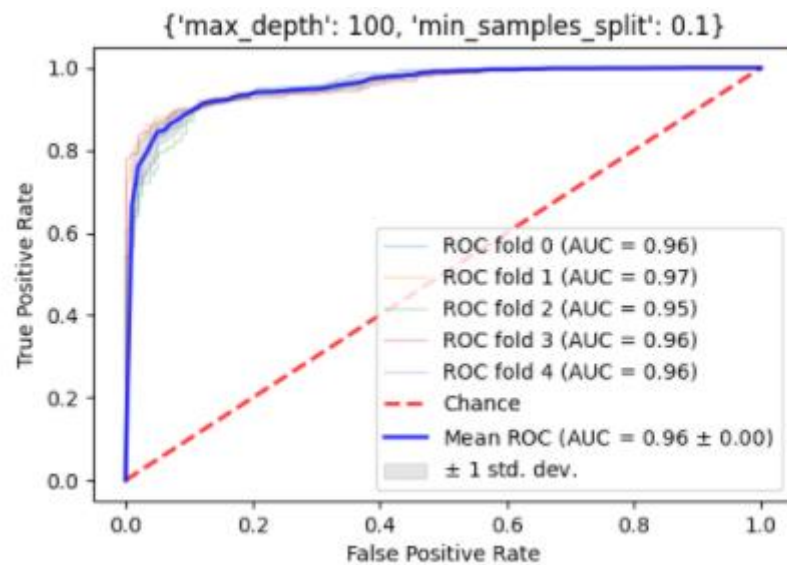
Hình 3.31 ROC và AUC của QDA



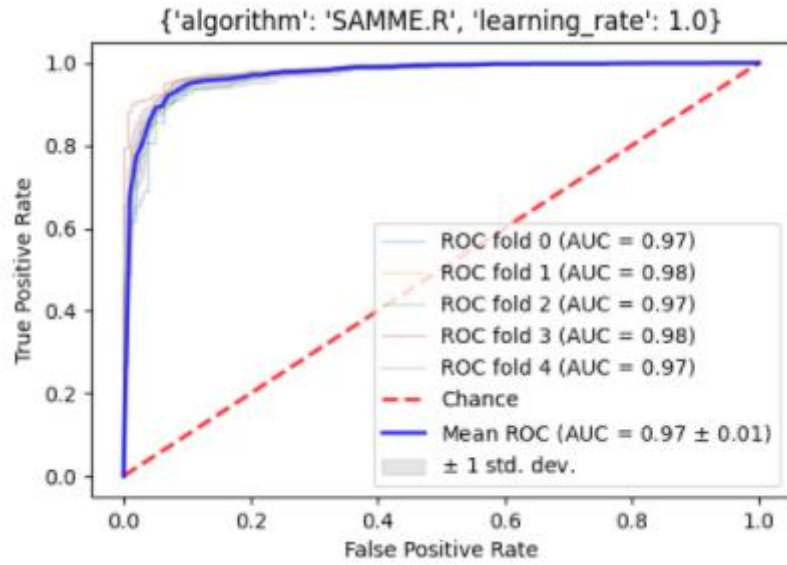
Hình 3.32 ROC và AUC của Decision Tree



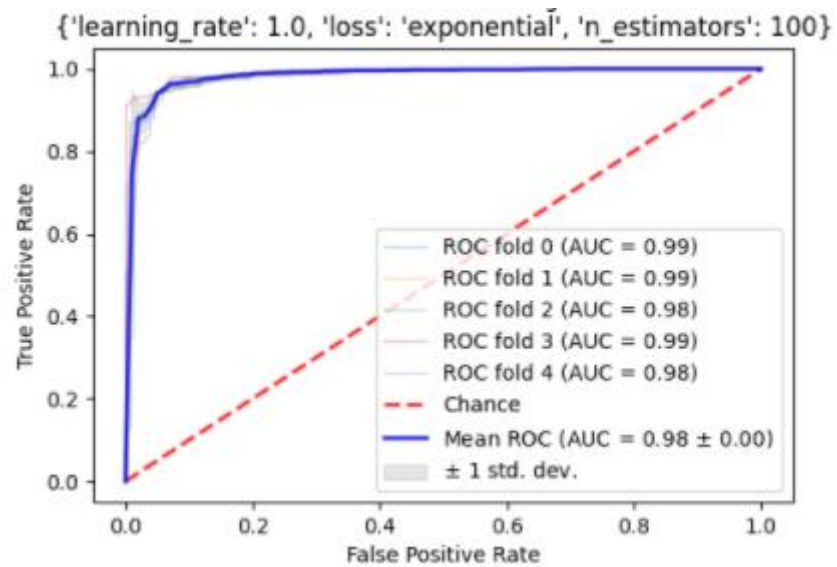
Hình 3.33 ROC và AUC của Random Forest



Hình 3.34 ROC và AUC của Extra Trees



Hình 3.35 ROC và AUC của AdaBoost



Hình 3.36 ROC và AUC của Gradient Boosting

Bảng đánh giá các chỉ số Precision, Recall, F1-score trên tập kiểm thử sử dụng siêu tham số đã hiệu chỉnh:

	precision	recall	f1-score	support
0	0.80	0.93	0.86	201
1	0.98	0.92	0.95	601
accuracy			0.92	802
macro avg	0.89	0.93	0.90	802
weighted avg	0.93	0.92	0.92	802

Bảng 3.16 Kết quả kiểm thử của KNN

	precision	recall	f1-score	support
0	0.94	0.81	0.87	201
1	0.94	0.98	0.96	601
accuracy			0.94	802
macro avg	0.94	0.90	0.92	802
weighted avg	0.94	0.94	0.94	802

Bảng 3.17 Kết quả kiểm thử của SVM

	precision	recall	f1-score	support
0	0.78	1.00	0.88	201
1	1.00	0.90	0.95	601
accuracy			0.93	802
macro avg	0.89	0.95	0.91	802

weighted avg	0.94	0.93	0.93	802
--------------	------	------	------	-----

Bảng 3.18 Kết quả kiểm thử của Logistic Regression

	precision	recall	f1-score	support
0	0.85	0.88	0.86	201
1	0.96	0.95	0.95	601
accuracy			0.93	802
macro avg	0.90	0.91	0.91	802
weighted avg	0.93	0.93	0.93	802

Bảng 3.19 Kết quả kiểm thử của Neural Network

	precision	recall	f1-score	support
0	0.38	0.87	0.53	201
1	0.92	0.53	0.67	601
accuracy			0.61	802
macro avg	0.65	0.70	0.60	802
weighted avg	0.79	0.61	0.64	802

Bảng 3.20 Kết quả kiểm thử của Naïve Bayes

	precision	recall	f1-score	support
0	0.73	0.85	0.78	201
1	0.95	0.89	0.92	601
accuracy			0.88	802

macro avg	0.84	0.87	0.85	802
weighted avg	0.89	0.88	0.88	802

Bảng 3.21 Kết quả kiểm thử của LDA

	precision	recall	f1-score	support
0	0.57	0.93	0.71	201
1	0.97	0.76	0.85	601
accuracy			0.81	802
macro avg	0.77	0.85	0.78	802
weighted avg	0.87	0.81	0.82	802

Bảng 3.22 Kết quả kiểm thử của QDA

	precision	recall	f1-score	support
0	0.67	0.80	0.73	201
1	0.93	0.87	0.90	601
accuracy			0.85	802
macro avg	0.80	0.83	0.81	802
weighted avg	0.86	0.85	0.85	802

Bảng 3.23 Kết quả kiểm thử của Decision Tree

	precision	recall	f1-score	support
0	0.93	0.68	0.79	201
1	0.90	0.98	0.94	601

accuracy	0.91			802
macro avg	0.91	0.83	0.86	802
weighted avg	0.91	0.91	0.90	802

Bảng 3.24 Kết quả kiểm thử của Random Forest

	precision	recall	f1-score	support
0	0.81	0.69	0.75	201
1	0.90	0.95	0.92	601
accuracy	0.88			802
macro avg	0.86	0.82	0.84	802
weighted avg	0.88	0.88	0.88	802

Bảng 3.25 Kết quả kiểm thử của Extra Trees

	precision	recall	f1-score	support
0	0.85	0.83	0.84	201
1	0.94	0.95	0.95	601
accuracy	0.92			802
macro avg	0.89	0.89	0.89	802
weighted avg	0.92	0.92	0.92	802

Bảng 3.26 Kết quả kiểm thử của AdaBoost

	precision	recall	f1-score	support
0	0.89	0.88	0.88	201
1	0.96	0.97	0.96	601

accuracy		0.94		802
macro avg	0.93	0.92	0.92	802
weighted avg	0.94	0.94	0.94	802

Bảng 3.27 Kết quả kiểm thử của Gradient Boosting

CHƯƠNG 4. ĐÁNH GIÁ HỆ THỐNG

4.1. Đánh giá mô hình

Qua kết quả huấn luyện các mô hình, ta thấy được thuật toán đạt hiệu quả cao nhất là Neural Network và GradientBoosting với AUC = 98%. Bên cạnh đó, các thuật toán khác cũng đạt AUC trên 95% là KNN, SVM, LDA, QDA, Random Forest, Extra Trees, AdaBoost. Các thuật toán còn lại đều cho kết quả tốt với AUC trung bình trên 0.96.

Do tập dữ liệu có độ lệch cao với tỉ lệ 1:3 (6013 mẫu nhãn 0 và 18035 mẫu nhãn 1) nên ta cần xét thêm các chỉ số Precision, Recall, F1-score để đảm bảo mô hình vẫn hoạt động hiệu quả. Do đặc thù bài toán là phát hiện mã độc trong một môi trường thực tế, và hạn chế nhất có thể việc bỏ sót các mẫu mã độc nên chỉ số false-negative (âm tính giả) cần phải càng thấp càng tốt.

Ta có:

$$Recall = \frac{TP}{TP + FN}$$

Giả sử số mẫu có nhãn positive là C , ta có thể biến đổi công thức trên để tính FN:

$$Recall = \frac{C - FN}{C - FN + FN} = \frac{C - FN}{C} = 1 - \frac{FN}{C}$$

$$\Leftrightarrow FN = (1 - Recall)C$$

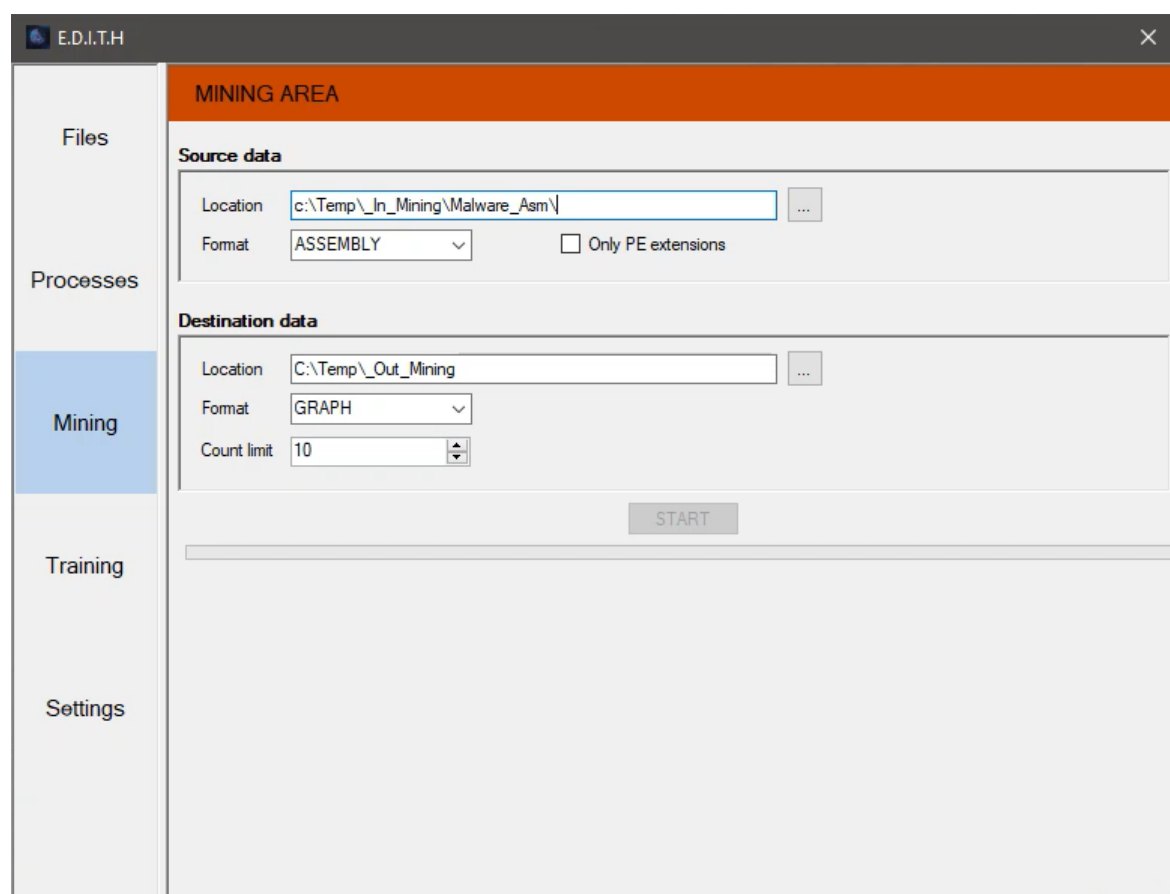
Trong đó C là hằng số nguyên dương, $Recall \in [0, 1]$ nên FN càng nhỏ khi Recall càng lớn. Theo kết quả trên, các thuật toán có AUC trên tập huấn luyện trên 95% và có Recall trên tập kiểm thử trên 90% là KNN, SVM, Neural Network, AdaBoost, Gradient Boosting. Điều đó chứng minh được các đặc trưng được trích xuất từ chuỗi mã lệnh đạt được độ chính xác, độ tin cậy cao và có khả năng ứng dụng vào thực tế.

4.2. Ứng dụng minh họa

Nhóm tác giả đã xây dựng được ứng dụng minh họa cho kết quả nghiên cứu của đề tài. Ứng dụng có các chức năng chính như sau:

4.2.1. Thu thập bộ dữ liệu

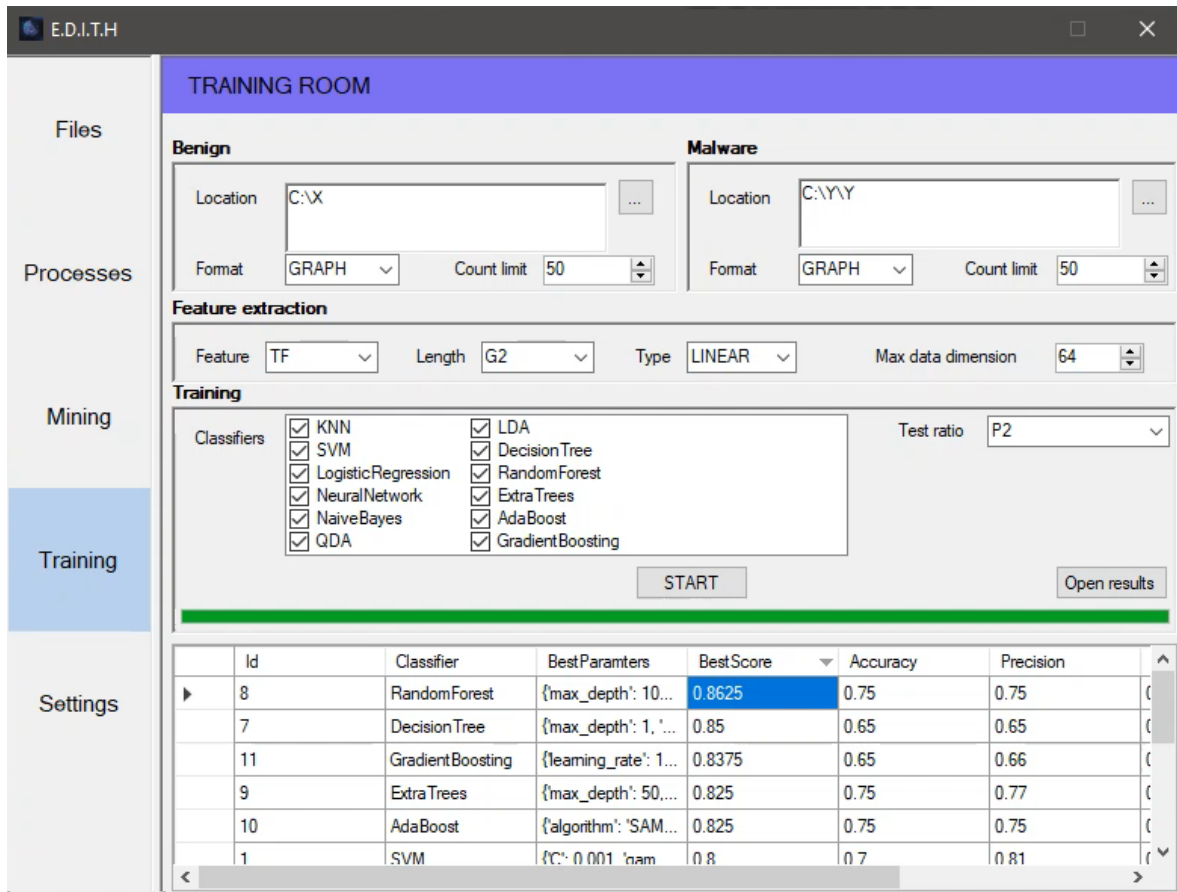
Đây là chức năng cho phép người dùng thu thập và rút trích những thông tin tối thiểu và cần thiết nhất từ các tập tin ở nhiều định dạng khác nhau như tập tin PE và các tập tin assembly nhằm chuẩn bị cho bước huấn luyện mô hình. Chức năng này cũng giúp người dùng tiết kiệm không gian lưu trữ cho bộ dữ liệu đầu vào, đồng thời giúp giảm thiểu thời gian rút trích đặc trưng.



Hình 37 Giao diện tính năng thu thập dữ liệu

4.2.2. Huấn luyện mô hình

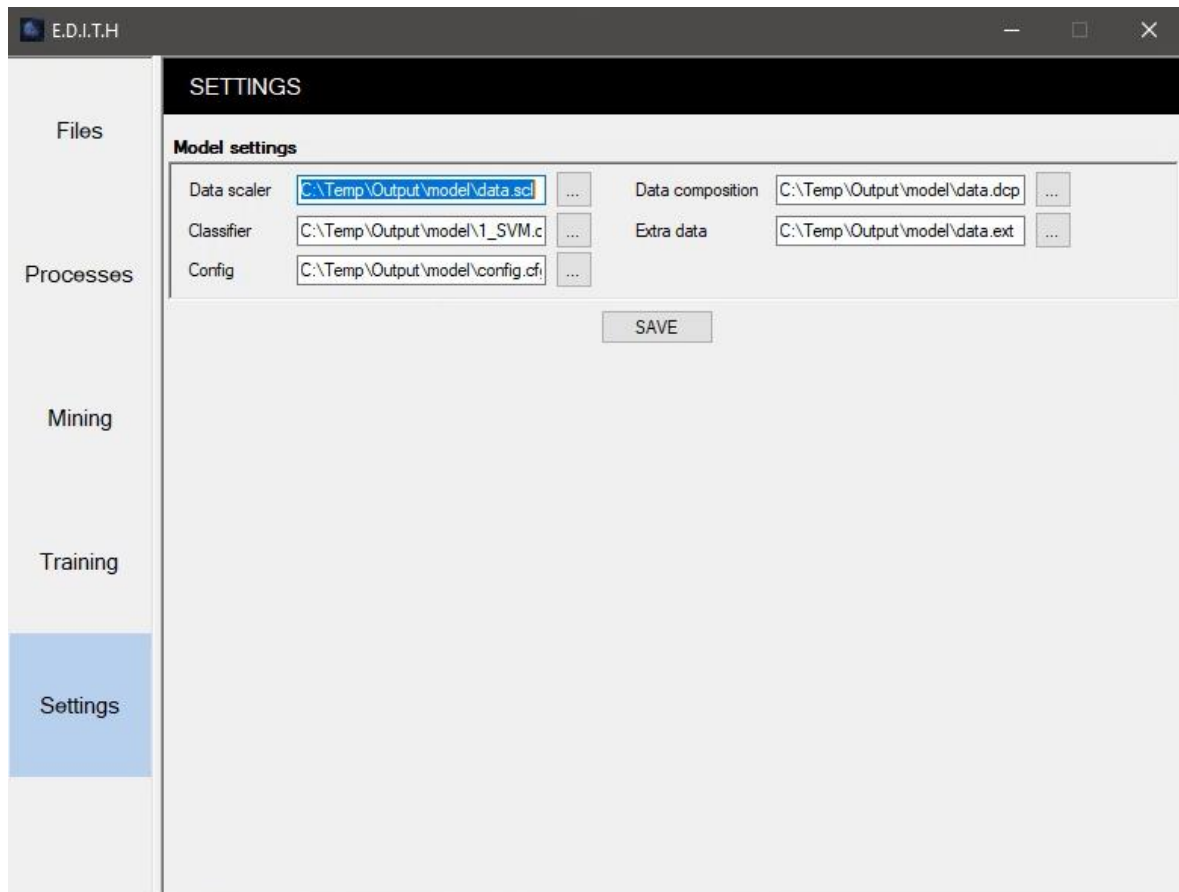
Chức năng huấn luyện hỗ trợ người dùng tự hiệu chỉnh các thông số và chọn bộ phân lớp cho việc huấn luyện. Chức năng này cũng đưa ra danh sách kết quả, thông số tốt nhất của từng bộ phân lớp nhằm giúp người dùng so sánh và đưa ra lựa chọn phù hợp.



Hình 38 Giao diện tính năng huấn luyện mô hình

4.2.3. Cài đặt mô hình cho ứng dụng

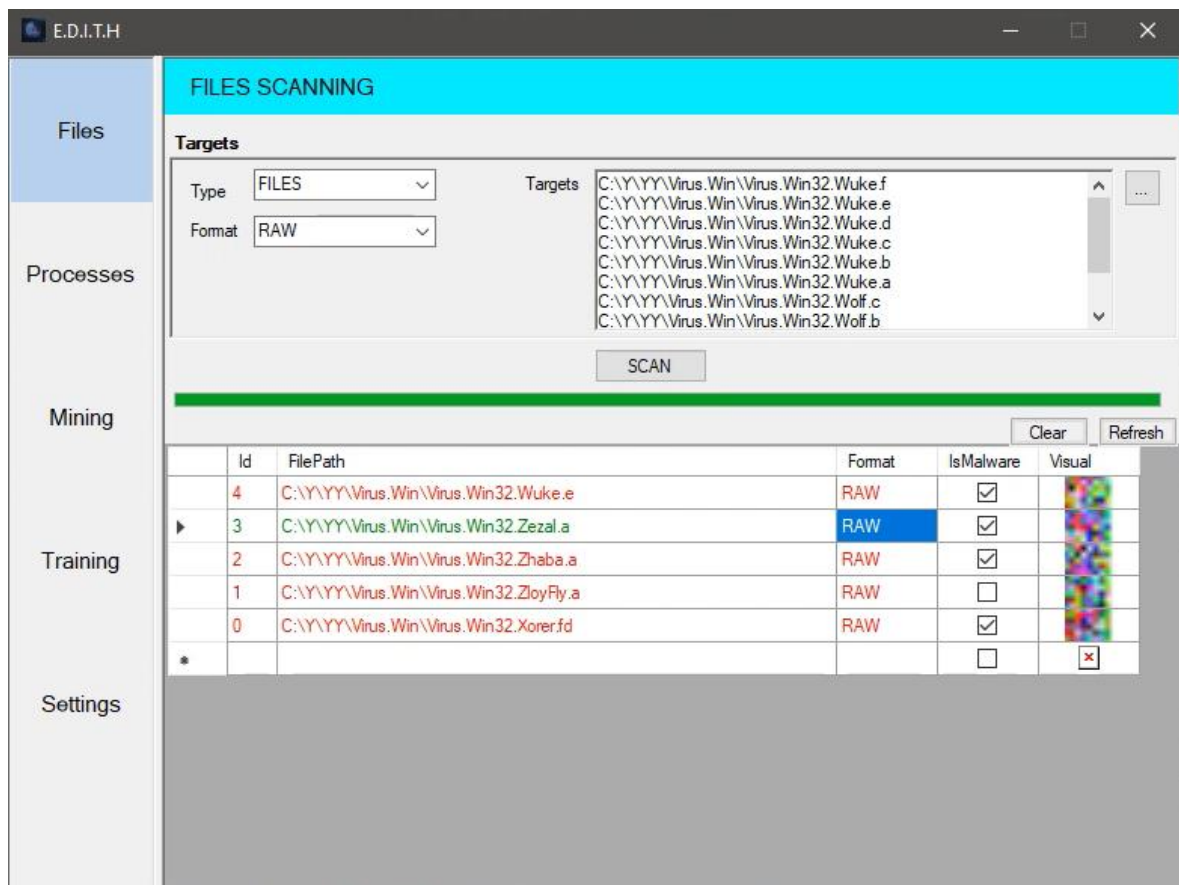
Chức năng này cho phép người dùng lựa chọn mô hình đã huấn luyện cho việc phát hiện mã độc.



Hình 39 Giao diện cài đặt

4.2.4. Quét các tập tin

Chức năng quét tập tin sẽ sử dụng mô hình do người dùng cài đặt để phát hiện những tập tin có khả năng mã độc.



Hình 40 Giao diện quét các tập tin

4.2.5. Quét các tiến trình

Chức năng quét các tiến trình sẽ sử dụng mô hình do người dùng cài đặt để phát hiện các tiến trình đang thực thi và sử dụng những tập tin PE độc hại.

E.D.I.T.H				
Files	PROCESSES SCANNING			
				Scan Refresh
	Name	PID	ExePath	Status
Processes	CAudioFilterAgent64	17012	C:\Program Files\CONEXANT\cAudioFilterAgent\CAudioFilterAgent...	BENIGN
	svchost	3440		UNKNOWN
	SecurityHealthService	8180		UNKNOWN
	vkise	9280	C:\Program Files (x86)\Comodo\Internet Security Essentials\vkise.e...	MALWARE
	svchost	2572		UNKNOWN
Mining	svchost	2136		UNKNOWN
	jusched	2996	C:\Program Files (x86)\Common Files\Java\Java Update\jusched.e...	MALWARE
	NisSrv	6256		UNKNOWN
	cis	2992	C:\Program Files\COMODO\COMODO Internet Security\cis.exe	BENIGN
	chrome	27904	C:\Program Files (x86)\Google\Chrome\Application\chrome.exe	BENIGN
Training	smss	404		UNKNOWN
	chrome	7664	C:\Program Files (x86)\Google\Chrome\Application\chrome.exe	BENIGN
	dragon_updater	4280		UNKNOWN
	svchost	6432		UNKNOWN
	Teams	46192	C:\Users\Home\AppData\Local\Microsoft\Teams\current\Teams....	BENIGN
Settings	ServiceHub.RoslynCode...	24936	C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise\Co...	BENIGN
	conhost	20716	C:\Windows\System32\conhost.exe	BENIGN
	svchost	2980		UNKNOWN
	svchost	12460		UNKNOWN
	svchost	4700		UNKNOWN
	svchost	8576		UNKNOWN

Hình 41 Giao diện quét danh sách tiến trình

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Trong đề tài này, nhóm tác giả đã thu thập thành công tập dữ liệu mã sạch bao gồm các tập tin hệ thống của hệ điều hành Windows 10 x64 1703, và trích xuất được thông tin từ hai bộ dữ liệu mã độc là “VxHeavens” và “Microsoft Malware Classification Challenge (BIG 2015)”. Đồng thời nhóm tác giả cũng xây dựng được mô hình máy học dựa trên các đặc trưng liên quan đến chuỗi mã lệnh của chương trình và đạt độ chính xác cao cũng như độ ổn định tốt.

6.2. Hạn chế và hướng phát triển

6.2.1. Hạn chế

Trong thời gian thực hiện khóa luận, nhóm tác giả chỉ mới thu thập các mẫu tập tin từ hệ điều hành Windows 10 thay vì các phiên bản hệ điều hành cũ hơn như Windows XP, Windows Vista, Windows 7, Windows 8. Các đặc trưng đã nghiên cứu và áp dụng trong đề tài còn khá ít và chưa tận dụng hết thế mạnh và thông tin từ mẫu đầu vào. Trong tương lai, đề tài có thể được tiếp tục khắc phục và hoàn thiện hơn.

6.2.2. Hướng phát triển

Trong tương lai, nhóm tác giả tiếp tục thu thập thêm mẫu dữ liệu phù hợp hơn, đồng thời cập nhật mô hình máy học định kỳ. Ngoài ra nhóm tác giả tiếp tục nghiên cứu và phát triển thêm nhiều đặc trưng hơn và nâng cao hiệu suất của hệ thống.

Mô hình máy học sẽ được đóng gói và lưu trữ, mã nguồn của toàn bộ quá trình huấn luyện sẽ được phát triển thành thư viện nhằm có khả năng tái sử dụng và tích hợp trên nhiều hệ thống khác nhau.

TÀI LIỆU THAM KHẢO

Tài liệu Tiếng Anh

- [1] Santos, I., Sanz, B., Laorden, C., Brezo, F., & Bringas, P. G. (2011). “Opcode-Sequence-Based Semi-supervised Unknown Malware Detection. Lecture Notes in Computer Science”, 50–57. doi:10.1007/978-3-642-21323-6_7
- [2] Sun, Zhi; Rao, Zhihong; Chen, Jianfeng; Xu, Rui; He, Da; Yang, Hui; Liu, Jie (2019). [ACM Press the 2019 2nd International Conference - Prague, Czech Republic (2019.03.15-2019.03.17)] Proceedings of the 2019 2nd International Conference on Geoinformatics and Data Analysis - ICGDA 2019 – “An Opcode Sequences Analysis Method For Unknown Malware Detection”, 15–19. doi:10.1145/3318236.3318255
- [3] Manavi, Farnoush; Hamzeh, Ali (2019). [ACM Press the Genetic and Evolutionary Computation Conference Companion - Prague, Czech Republic (2019.07.13-2019.07.17)] Proceedings of the Genetic and Evolutionary Computation Conference Companion on - GECCO '19 – “A new approach for malware detection based on evolutionary algorithm”, 1619–1624. doi:10.1145/3319619.3326811
- [4] Anh Viet Phan; Minh Le Nguyen; Lam Thu Bui. “Convolutional Neural Networks over Control Flow Graphs for Software Defect Prediction”. 2017 IEEE 29th International Conference on Tools with Artificial Intelligence (ICTAI). doi:10.1109/ICTAI.2017.00019
- [5] Microsoft Corporation. “Microsoft Portable Executable and Common Object File Format Specification”. 1999
- [6] “VX Heaven windows virus collection”.
<https://archive.org/details/vxheaven-windows-virus-collection>
- [7] “Microsoft Malware Classification Challenge (BIG 2015)”.
<https://www.kaggle.com/c/malware-classification>

- [8] “Capstone engine”. <http://www.capstone-engine.org/>
- [9] “An In-Depth Look into the Win32 Portable Executable File Format”.
<https://docs.microsoft.com/en-us/archive/msdn-magazine/2002/february/inside-windows-win32-portable-executable-file-format-in-detail>
- [10] “PE Format”. <https://docs.microsoft.com/en-us/windows/win32/debug/pe-format>

Tài liệu Tiếng Việt

- [11] Vũ Hữu Tiệp. “Machine Learning cơ bản”
- [12] Lê Minh Hoàng. “Giải thuật và lập trình”.